anabrid

lucidac handbook

Serial code sticker

Device identification
and authentication

Here you can attach copies of the labels which are permanently put on the back of the LUCIDAC corresponding to this manual. This helps finding relevant data quicker.

Never connect this device directly to the main power line. Do not apply voltages greater than ± 2 V to this device's front connectors. The application of voltages greater than ± 2 V to this device's front panel connectors may cause damage to property, personal injury, or death. Only connect the supplied power supply to the power socket on the back of the device.
This device is designed for users aged 12 years and older. Users below the age of 12 require adult supervision.

LUCIDAC User Manual

Mar 2026, V. 5.0 (fifth edition)

anabrid GmbH,
Am Stadtpark 3,
12167 Berlin, Germany.
Phone: +49 30 6293047 20
Email: hello@anabrid.com
Web: www.anabrid.com

You can read the latest version of this document online at
https://anabrid.com/lucidac-user-manual.pdf

Contents

Preface **1**

1 Introduction and Getting started **3**

1.1 Definition of technical terms 3
1.2 Requirements 5
1.3 What is in the box 6
1.4 First time hardware setup 7
1.5 Device connectivity 8
1.6 The LUCIDAC co-processor design 9
1.7 Installing the *pybrid* reference code 11
1.8 Software stacks and openness 12

2 LUCIDAC architecture **15**

2.1 Interconnection network 15
2.2 LUCIDAC Blocks 19
2.3 Math blocks 20
2.4 Circuit Configuration 21
2.5 Coupling multiple LUCIDACs together 23
2.6 Coupling THAT with LUCIDAC 26

3 Tutorial: A first model on the LUCIDAC **27**

3.1 Connecting to your LUCIDAC 27
3.2 Using the lucipy-interface from Python 28
3.3 Using the pybrid CLI client 32

4 Example applications **35**
4.1 Lorenz attractor . . . 36
4.2 Rössler attractor . . . 40
4.3 Hindmarsh Rose Neuronal Bursting . . . 43
4.4 Van der Pol oscillator . . . 46

5 Device administration and maintenance **48**
5.1 Detection and monitoring utilities in `pybrid` . . . 48
5.2 Device identification . . . 49
5.3 Firmware Update . . . 49
5.4 Physical maintenance . . . 50
5.5 Testing and calibration . . . 50

6 Embedded programming **51**
6.1 Front panel digital connector . . . 53
6.2 Software architecture and communication interface . . . 54
6.3 System states . . . 55
6.4 Writing a compile time plugin . . . 57
6.5 Extending the existing network protocol . . . 57

7 Troubleshooting **59**
7.1 Basic connectivity and startup . . . 59
7.2 Analog programming problems . . . 61
7.3 Frequently asked questions (FAQ) . . . 62

8 Resources and further reading **64**

9 Legal documents **66**
9.1 European Union CE/RoHS Conformity Declaration . . . 66
9.2 Safety Instructions, Maintenance, Warranty and Liability . . . 67

Preface

Welcome to Analog Computing

Analog computers differ substantially from current digital computers in that they do not work by executing an algorithm in a step-by-step fashion. Instead they consist of a number of *computing elements*, each capable of performing a certain mathematical operation such as summation, multiplication or time-integration. A *program* for an analog computer describes how these computing elements are to be connected in order to create a *model* (an *analogue*) of the problem to be solved.

A (very) simple problem like computing $a(b + c)$ could be implemented on a classic digital computer as shown in figure 1. This straightforward algorithm requires six individual steps to compute the desired result. Contrast this with the analog computer program shown in figure 2. This setup requires just two computing elements, one summer and one multiplier. The summer is fed with the values b and c while the multiplier is connected to the output of this summer and to the value a.

```
LOAD  A, R0
LOAD  B, R1
LOAD  C, R2
ADD   R1, R2, R1
MULT  R0, R1, R0
STORE R0, ...
```

Figure 1: Computing $x = a(b+c)$ on a digital computer

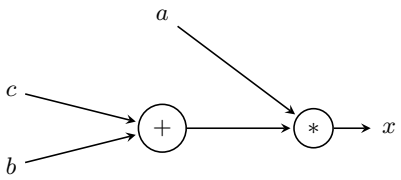


Figure 2: Analog computer setup for solving $x = a(b + c)$

Typically, values are represented by voltages or currents in an analog computer so that only a single connection is necessary between computing elements.

The advantages of this analog computing approach are manyfold. Most notable are the extremely high degree of parallelism (there are no central memory, no data dependencies, no synchronisation points, etc., so that the computing elements work in full parallelism), the resulting high speed of computation and the inherent very high energy efficiency of analog computing.

While a digital computer can basically solve every problem given enough time and memory, an analog computer setup needs as many computing elements as there are operations in the governing equations of the problem to be solved. A little bit more mathematically, one can say that the size of a digital computer is constant while its time to solution typically grow much faster than just linearly with the size of the problem (making many problems basically intractable on classic digital computers). The size of the analog computer on the other hand grows linearly with the problem size but, and this cannot be overestimated, the time to solution remains constant.

Classic analog computers were impressive systems, typically featuring a large patchpanel with thousands of jacks, all connected to a vast number of computing elements, such as integrators, summers, coefficient potentiometers, multipliers, etc. Programming these machines was as much of an art as a science and was quite time consuming due to the hundreds or even thousands of connections that had to be made manually. ([Ulmann, 2023/2] describes the history of analog computing in great detail.)

Nowadays, the patchpanel is a museum piece and the actual connection of the computing elements is done electronically, under the control of an attached digital computer; this greatly simplifies the programming. Using appropriate libraries as shown below, the analog computer can be used as a mathematical machine without having to understand the underlying electronic implementation in detail.

Since the basics of analog computer programming are outside the scope of this user guide, please refer to [Ulmann, 2023] for detailed information on the subject.

Section 1

Introduction and Getting started

The LUCIDAC is a fully software-reconfigurable analog-digital hybrid computer intended to be used as a form of co-processor in conjunction with a digital computer. Its main area of application is the exploration of analog and hybrid computing approaches in a variety of fields including high-performance computing, life sciences, mathematics, hardware-in-the-loop setups for (industrial) control purposes, education and training, and many more.

LUCIDAC is the first commercially available modern hybrid computer, and has eight integrators with two software selectable time scale factors of $k_0 \in \{10^2, 10^4\}$, four four-quadrant multipliers, 32 coefficient elements with 11 bits of resolution plus a sign bit each, and a variable number of (implicit) summers. Using these computing elements it is possible to solve eight coupled differential equations (*DEQ*s) of 1^{st} order or four DEQs of 2^{nd} order, etc. Thanks to the multipliers, non-linear equations can also be easily implemented.

The LUCIDAC system contains a local microcontroller unit (*MCU*) responsible for the communication with the attached digital computer, configuring the computing elements and their interconnection, reading out values by means of analog-digital-convertes (*ADC*s), etc.

1.1 Definition of technical terms

The following terms are used either in their full or abbreviated versions within this document:

ADC **A**nalog-**D**igital **C**onverter. A component that converts LUCIDAC's analog voltage signals into digital values for data acquisition and processing by the embedded microcontroller.

DAC **D**igital-**A**nalog **C**onverter. A component that converts digital values from the microcontroller into analog voltage signals used by LUCIDAC's computing elements.

DAQ **D**ata **AcQ**uisition subsystem. The LUCIDAC's built-in system for capturing and digitizing analog computation results using internal ADCs, useful for hybrid programming applications.

DSO **D**igital **S**torage **O**scilloscope. An external measurement device recommended for visualizing LUCIDAC's analog output signals; can be connected via the MCX output channels using the supplied cables.

MCX **M**icro **C**oa**X**ial connector. A compact coaxial connector type used on LUCIDAC's front panel for all analog I/O channels, digital status lines, and signal generator outputs; operates within a ± 2V voltage range.

BNC **B**ayonet **N**eill-**C**oncelman connector. A common coaxial connector standard used on many oscilloscopes and test equipment; LUCIDAC includes MCX-BNC cables for connecting to external instruments.

MCU **M**icro**C**ontroller **U**nit. The real-time capable embedded microprocessor inside LUCIDAC that controls the analog computing elements and communicates with the upstream computer via Ethernet or USB.

CPU **C**entral **P**rocessing **U**nit. The main processor of the external computer (notebook, desktop, or server) that runs client software like lucipy to program and control the LUCIDAC system.

GND **G**rou**nd** reference. The common electrical reference point (0V) for all LUCIDAC signals; essential for proper signal measurement and preventing damage to the analog computing elements.

RCA RCA connector (**R**adio **C**orporation of **A**merica). A type of electrical connector; in LUCIDAC context, may refer to alternative connection methods though MCX is the primary standard used.

ESD **E**lectro-**S**tatic **D**ischarge. A sudden flow of electricity that can permanently damage LUCIDAC's sensitive analog components; proper grounding and handling precautions are essential when connecting external equipment.

1.2 Requirements

The LUCIDAC system is designed for ease of use. However, the user should have basic programming, ideally some experience with (scientific) Python, as this is the main software environment used here. Furthermore, some math knowledge is required (calculus, differential equations). The minimum hardware requirements are:

- 100 V – 240 V AC mains power connection for the supplied power supply.
- A notebook, desktop or server grade computer with ordinary IPv4 networking and/or a USB2 interface. If the USB interface is to be used, root/administrator rights are very likely to be required.
- The operating systems Apple Mac OS X™ (macOS 10.9 Mavericks or later), Microsoft Windows™ (Windows 7 or later) or GNU/Linux are supported. In addition to this at least Python 3.11 (released in 2022) is required for the reference client software *pybrid*.

It is recommended to use the device in a conventional "local" network with a DHCP server and networking switch. Furthermore, a digital storage oscilloscope (*DSO*) is recommended for convenience. However, note that the following points are not mandatory in order to use LUCIDAC:

- Operating the LUCIDAC system does not require an Internet connection. The LUCIDAC firmware will never intentionally connect to another host if not instructed to do so. The LUCIDAC will never "call home" without being told so.
- Connecting LUCIDAC over USB does not require USB-C or USB3 on the host computer. Classical USB 2.0 is sufficient.
- A conventional local network is not needed but is strongly recommended if connecting via TCP/IP. Technically, connecting to the LUCIDAC over TCP/IP neither requires a DHCP server or networking switch. Experts can configure the LUCIDAC can with a static IPv4 address through firmware modification (see chapter 6).
- LUCIDAC provides internal methods for data acquisition which come in handy with hybrid programming, reaching up to 500,000 samples per second. However, using a DSO provides better user interaction, especially initially and is the natural paradigm for *continuous signals*.

1.3 What is in the box

The LUCIDAC system is shipped with the following complement of items:

1. The LUCIDAC itself, housed in a durable and easily stackable metal case (due to the low power consumption no active ventilation/cooling is required).
2. A universal power supply with a 24 V DC output. Please note that the user will need to provide a country-specific mains lead with an IEC C13 connector.
3. A collection of MCX-MCX and MCX-BNC cables for interfacing the LUCIDAC system to external signal sources, control equipment, oscilloscopes, etc.
4. A digital port header as well as a suitable long flat ribbon cable for connecting multiple LUCIDACs together.
5. An RJ45 Ethernet cable suitable for connecting the LUCIDAC system to your inhouse network or to your computer (cross-over).
6. An USB-C to USB-A cable suitable for connecting the LUCIDAC system directly to your computer.
7. This user guide or, in newer versions, a QR code linking to resources on the internet, including this guide.

1.4 First time hardware setup

We recommend connecting the LUCIDAC to an existing ethernet network with a DHCP server. The LUCIDAC obtains its IP address via DHCP. Ensure your end device (notebook, desktop or server) is part of the same IP network. To find the LUCIDAC's IP address, use the `pybrid detect` command (see Section 3.1). Alternatively, the LUCIDAC can be *autodiscovered* automatically by pybrid within the same multicast (broadcast) domain.

Due to the high sample rate and low latency, the Ethernet connection is generally preferered over an USB connection, the latter only being used for firmware updates and embedded programming. The recommended steps are as follows:

1. Connect to Ethernet.

2. If using a DSO, connect the first few LUCIDAC outputs channels to your DSO inputs using the enclosed MCX-BNC cables. If desired you can use the LUCIDAC OP output (this line represents the mode of operation of the LUCIDAC and is a the main trigger source for external deivces) as the trigger input for the DSO.

3. Power the LUCIDAC device on by flipping the power switch on the front.

The LUCIDAC system boots within about 10 seconds. The 8 LED array will light up and the system is fully booted when the LEDs turn off again. Note that the system is passively cooled and thus fan less and completely silent. In order to turn off the system again, just flip the power switch. There is no need to perform any shut down procedure.

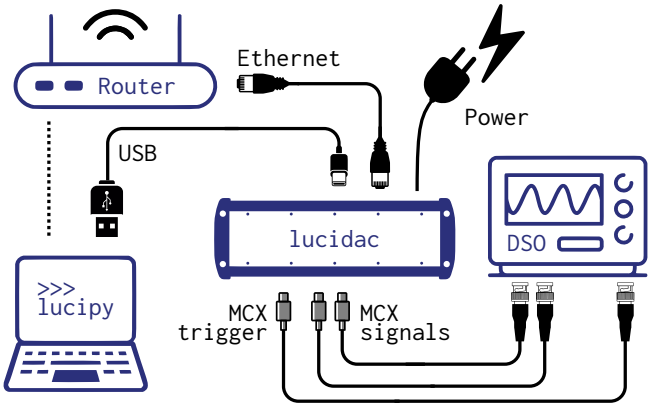


Figure 1.1: Recommended first time setup

1.5 Device connectivity

The LUCIDAC front (Figure 1.2) has a variety of analog and digital inputs and outputs, while the back panel (Figure 1.3) has power and communication ports.

The digital part of the front panel has a 3.3V logic level, regardless of the MCX or pin connector shape. The four binary status lines, available at their respective MCX connectors, are shown in table 1.1.

The analog I/O signals have a ± 2V volage range which translates to the ± 1 machine unit domain of the system. The eight output channels are connected to particular system matrix *lanes* and are typically used for connecting external measurement equipment, such as a DSO. The eight input channels can be used as inputs to the system matrix, each replacing one lane. (This will become clearer in section 2 when the LUCIDAC architecture has been described.)

The right side of front panel contains four MCX *outputs* connected to the built-in signal generator. These signals can be either connected to MCX inputs on the front panel or used for other purposes. *Do not* short circuit or overload these outputs, otherwise the signal generator will be damaged.

Never connect sources exceeding ± 2V to the MCX front panel jacks. This will permanently damage the computing elements!
The inputs and outputs can be damaged by excessive external signal amplitudes. The same is true for the pin header. Using the input/output ports requires a thorough understanding of the signal levels involved.

1.6 The LUCIDAC co-processor design

The LUCIDAC achieves the goal of an analog co-processor with the help of an embedded microprocessor which is connected via ethernet to an upstream computer (Figure 1.4). This creates a heterogenous computing environment with a “big” CPU which is relatively far away from the actual LUCIDAC system (also typically not real-time capable, both in terms of the connectivity as well as with respect to the operating system used) and a “small” MCU which is real-time capable but has relatively low computing power.

Exploiting the performance advantages of analog-digital hybrid algorithms requires clever distribution of an algorithm between the CPU, the MCU, and the analog computing elements of the LUCIDAC. This topic is discussed in more detail in section 6. A simpler way of programming such a hybrid computer, referred to as *client based programming*, is described first. Depending on the connection between CPU and MCU, different names for the roles of MCU and CPU codes are commonly used:

1. When using ethernet, a classical *client-server model* is used. In this setting, the embed-

IC	**OP**	**OL**	**HALT**
Initial Conditions	Operating	Overload	External halt
The system is in preparation mode and the integrators load their initial conditions. This is normally an *output* signal (see section 2.5 about master/minion mode for alternative modes).	The system is in computing mode and the integrators are operating. This, too, is normally an *output* signal.	An overload is detected in the computing circuit. This is always an *output*. Depending on the current configuration of the system, this condition can be used to automatically halt the current computation.	This is an *input* signal and allows halting the machine controlled by an external, incoming trigger signal.

Table 1.1: Front panel system mode LEDs and MCX sockets and their descriptions

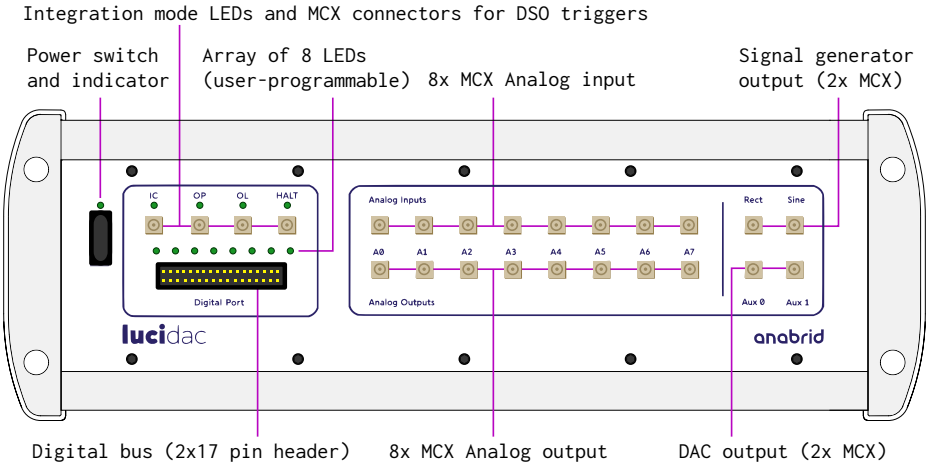


Figure 1.2: Front connectivity

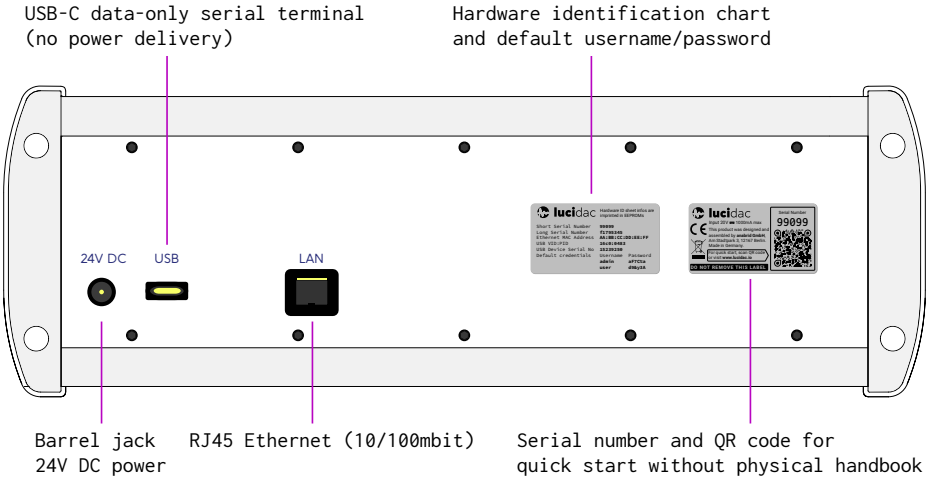


Figure 1.3: Back connectivity. For type plates (labels) see also front matter

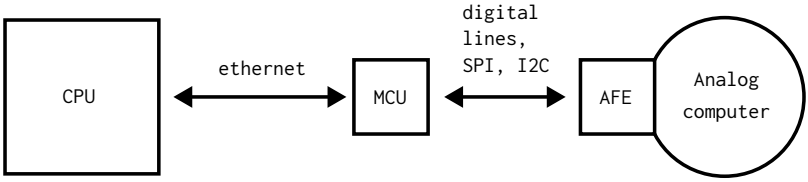


Figure 1.4: CPU (central processing unit)/MCU (microcontroller unit) concept. AFE denotes the analog front-end. SPI and I2C are serial protocols mainly used for communication between integrated circuits.

ded microcontroller within LUCIDAC acts as a network-enabled *server*. A variety of different *client* codes exist which run on the CPU. Multiple clients can connect to one server at the same time. LUCIDAC allows locking the device to a single client.

2. When using USB, the USB *host computer* runs the "client code" with the MCU acting as USB *peripheral*. There can always be only one "client" be connected via USB. Ethernet and USB can be used at the same time.

In any situation, one CPU/client can be connected to multiple LUCIDACs at the time, regardless of the transport method (Ethernet or USB).

1.7 Installing the *pybrid* reference code

For the LUCIDAC, client code implementations are available for Python in the form of the *pybrid* package (PyPi: `pybrid-computing`). The communication protocol with the device is open source, hence clients in other languages may beautiful added by developers and users. Here we focus on *pybrid*, which is anabrid's implementation of a full stack for hybrid computing. In order to simplify the entrance into the world of hybrid and analog computing, we provide to *lucipy* ("LUCIdac on PYthon") interface offering a simple interface for defining and operating circuits on your LUCIDAC. For the remainder of this manual, *lucipy* will thus serve as a synonym referring to the lucipy-interface within pybrid.

It requires at least Python 3.11 and thus should run on any (popular) computer platform built in recent years. We recommend using `uv` (`https://docs.astral.sh/uv/`), a fast Python package manager, for installation. pybrid can be installed with the following commands using `uv`:

```
shell@client $> uv venv --python 3.13
shell@client $> uv pip install pybrid-computing
```

Extensive documentation is available at https://anabrid.github.io/pybrid-computing/. The code features:

- a simple syntax to setup and edit LUCIDAC circuit configurations on a netlist level.
- A client interface to steer the LUCIDAC operations, including data acquisition.
- A set of examples showing how to implement mathematical models on the device, see also chapter 4.

Pybrid offers functions to detect the IP address of a LUCIDAC device (see section 5). If this fails, you have to tell lucipy how to reach the system. To do so, lucipy used a notation called *endpoints*. These are similar to a VISA lab device connection string. LUCIDAC endpoints are written similar to URLs used in the web. See the lucipy manual for further instructions. Section 7.1 contains a list of tips and tricks if you should encounter problems connecting to your LUCIDAC system.

Lucipy abstracts only slightly from the underlying hardware. Computing elements are connected manually, guided by the underlying system of differential equations to be solved. This is quite similar to programming a digital computer in assembler. The example circuits in chapter 4 demonstrate how it is done. For a more detailed, step-by-step introduction, please refer to the tutorial in chapter 3. An in-depth explanation of how to map a mathematical problem to an analog computer circuit is outside of the scope of this handbook. For more details on this refer to the references given in section 8.

1.8 Software stacks and openness

The LUCIDAC was originally released with a fully open-source software stack under the MIT License, consisting of the *lucipy* Python client (https://github.com/anabrid/lucipy) and the LUCIDAC firmware (https://github.com/anabrid/lucidac-firmware). This “initial stack” remains fully functional and can be freely used, modified, and distributed under the terms of the MIT License. However, it is **incompatible** with anabrid’s current mainline software stack due to fundamental protocol differences (JSON vs. Protocol Buffers). The initial stack will

not be actively maintained by anabrid, but the permissive license ensures that users retain full control over these components.

For users who are not developers, we recommend the mainline stack which offers more features, better stability, and ongoing support. The simple circuit-definition syntax originally provided by the standalone *lucipy* package has been integrated into the *pybrid* package. For the remainder of this manual, “lucipy” refers to this syntax within pybrid, and “firmware” and “stack” always refer to the mainline setup.

For downloads of software, binaries, and firmware, please visit `https://github.com/anabrid/lucidac-resources`.

Section 2

LUCIDAC architecture

Classic analog computers typically featured a large central patch panel consisting of thousands of sockets by means of which computing elements were connected with each other using patch cables. This was very cumbersome, took a long time to manually program, was error prone, and did not allow for rapid program changes. Fortunately with the LUCIDAC system these patch panels are finally relegated to museums where they belong.

> Do not confuse the LUCIDAC front panel connectors with the patch panels of classical analog computers. The LUCIDAC front panel serves only for analog and digital input and output (I/O), while the computing elements are interconnected within the enclosure.

2.1 Interconnection network

LUCIDAC provides all-to-all connectivity between up to 16 computing elements. Coupling is implemented by switching matrices under control of the attached digital computer, basically resembling a *crossbar switch*.

Schematically, Figure 2.1 provides a way to visualize the interconnection matrix with real numbers representing coupling strengths. The LUCIDAC system matrix is the adjacency matrix for the interconnection graph between the computing elements. The entries are referred to as *weights*. As a special property, the system matrix does *implicit summing* which can be thought of as matrix multiplication linear algebra operations: $C_i^{\text{in}} = \sum_{j=0}^{16} M_{ij} C_j^{\text{out}}, \quad i,j \in [0,16]$ where C_i^{in} is the input to the i-th computing element, C_i^{out} is its output, and the M_{ij} are the

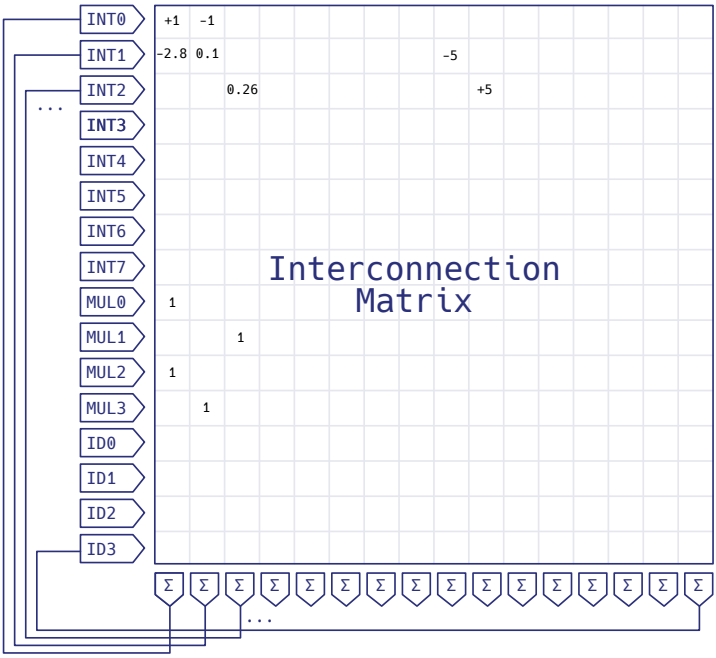


Figure 2.1: The LUCIDAC system reduced to its system matrix. The entries shown are for the Lorenz system (section 4.1)

weights. The previous equation describes *monadic* computing elements with one input and one output. The integrator is the only monadic computing element in a LUCIDAC system. It computes the time integral over its time-varying input signal. LUCIDAC also provides another kind of computing elements, which are *dyadic*, i. e. feature two inputs and one output: Multipliers. The multipliers implemented allow for full four-quadrant-multiplication, i. e. each of the input values can take on values within the whole machine unit interval $[-1, 1]$.

An important property of the LUCIDAC interconnection is that it is *CTCV* (continuous time, continuous variable). This means that the connections between computing elements within a LUCIDAC system are not based on switched capacitors but instead on static (reconfigurable) switching matrices. Although a switched capacitor network could have simplified the actual hardware implementation, it would impose an artificial cutoff frequency on the overall analog computer setup.

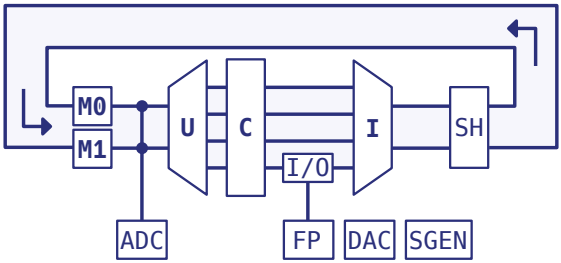


Figure 2.2: The LUCIDAC system as a closed-loop feedback circuit (“turbine diagram”)

Figure 2.2 shows a more detailed block diagram of the LUCIDAC. It shows the closed loop *analog compute path*. The labelled shapes in the diagram each represent a particular *blocks* (in bold letters, referred to as “entities” in the LUCIDAC terminology) and auxiliary *elements* (in normal letters). The figure does not show any digital control/configuration signal paths. Most notably, the **U**-, **C**- and **I**-blocks together form the **UCI** matrix, which corresponds to the simplified *interconnection matrix* shown before in Figure 2.1. In contrast, Figure 2.2 emphasizes the internal *fanout* and *fanin* properties of the U- and I-blocks, resulting in a turbine-like appearance of this circuit representation.

The UCI matrix has 16 inputs, 16 outputs and 32 internal signal paths, each of which contains a coefficient element. These 32 paths are also called *lanes* and correspond to up to 32 nonzero entries in the system matrix, yielding a maximum matrix density of $32/16^2 = 12.5$. This corresponds to a minimum sparsity of 87.5%.

Figure 2.3 shows the most detailed block diagram within this document. Here, the internals of the different blocks are sketched and individual analog data lines are drawn, communicating one voltage or current, respectively. A textual description about the individual blocks is given in the following pages.

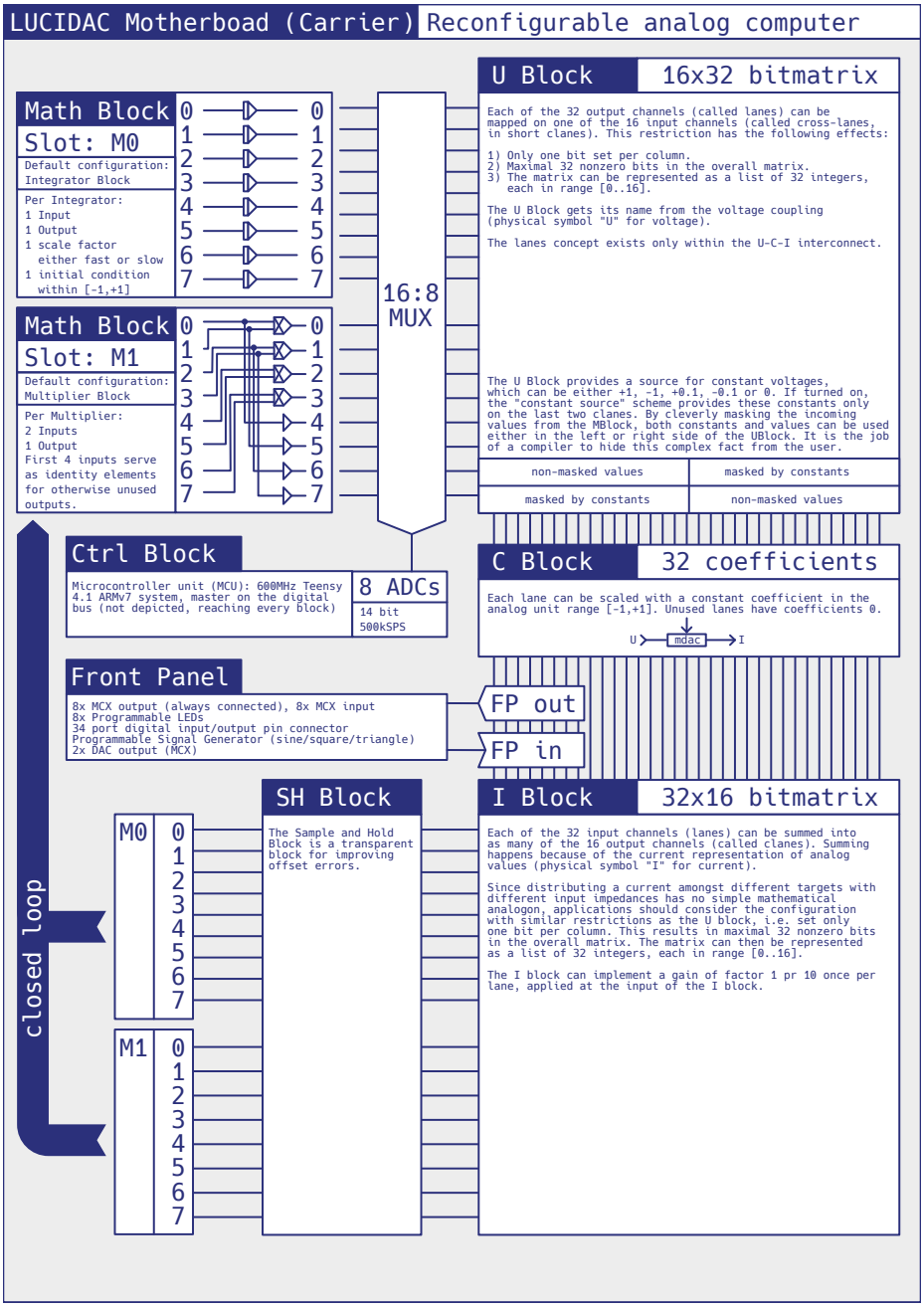


Figure 2.3: LUCIDAC detailed architecture

2.2 LUCIDAC Blocks

`M0` and `M1` blocks: M-blocks are *math blocks*. Each can have up to 8 analog input and 8 analog output signals. Math blocks contain various elements and allow digital configuration of these elements. In contrast to classic analog computers, there are no summers available as explicit computing elements, as summation is done implicitly in the I-block. This enhances the overall flexibility considerably. For details about the Math blocks, see Section 2.3.

`U` block: The output signals of these M-blocks are connected to the U-block which contains a 16×32 crossbar switch. Since the output signals of the M-blocks are voltages, this crossbar switch can distribute one signal to several outputs at once. It therefore serves as a 16:32 fan-out, allowing to distribute the input signals arbitrarily on 32 internal lanes within the UCI interconnection matrix.

`C` block: These 32 output signals (still represented by voltages) are then fed into the *coefficient block*. This contains 32 digitally controlled coefficient potentiometers with 11 bits resolution and an additional sign bit. Thus, the C-Block implements one coefficient per lane by means of a multiplying DAC (a certain type of digital analog converter). These coefficients allow scaling of values with factors in the interval $[-1, 1]$. The output of these coefficients are now currents instead of voltages and feed the I-block.

`I` block: Following the C-block is the I-block. It, too, is a crossbar switch, this time with 32 inputs and 16 outputs. Working with currents, it is now possible to implicitly sum several inputs, thus eliminating the need for explicit summers as computing elements. The 16 output lines of the I-block are connected to the inputs of the computing elements contained on the M-blocks. The I-block also features a programmable gain of either factor 1 or 10, allowing for a broader dynamical range of the machine.

`SH` block: This block contains *sample and hold* elements to correct offset errors of the computing elements. It is transparent for the analog signal path and only serves for signal conditioning. It is part of the sophisticated error correction techniques of the LUCIDAC.

`CTRL` block: This block contains the *hybrid controller* based on the MCU. It is responsible for setting up the computing elements, coefficient potentiometers, crossbar switches. It also takes care of the communication with the digital upstream computer (the *client* in a networking setting or *host* in a USB setting).

The Control block is not part of the compute path and therefore is not shown in figure 2.2. The following additional *elements* are shown:

ADC: For device-internal data aqusition (DAQ), there are eight 16 bit analog-to-digital converters (ADC). They have access to all 16 M-block outputs by means of a separate 16:8 analog multiplexer.

I/O and FP: The Front panel allows to connect eight analog input and eight output signals. The last eight of the overall 32 lanes are connected to these front panel output jacks. The input signals can be fed into a LUCIDAC setup by means of eight analog switches. In some parts of the documentation these signals are called `ACL_IN/OUT` (short for Analog Cluster In/Out).

DAC and SGEN: The LUCIDAC also features a signal generator (SGEN) for square/triangle/sine signals with adjustable frequency as well as two digital-to-analog converters (12 to 16bit DAC). Outputs from the SGEN/DACs can be connected to the LUCIDAC computing elements by means of MCX-MCX cables.

2.3 Math blocks

Apart from the implicit summing in the networking topology, all linear and nonlinear analog computation is performed on the Math blocks (M-blocks). LUCIDAC math blocks are modular and can be exchanged (although this requires opening the device, see Section 5.4). LUCIDAC has slots for two math blocks. As of now there are two types of M-blocks available and the machine is by default equipped with one of each type:

Math block `M0` contains eight integrators (each with one input with implicit summing, one output). Integrators have an internal analog state (the current integration value), a digital state (IC / OP / HALT state machine), and a hybrid state (initial conditions and time scaling factor k_0). **Note that LUCIDAC's integrators invert during integration, i.e. they compute** $-\left(\int_0^t (\sum_i I_i) - e_0\right)$ where e_0 is the initial condition and I_i are the input values to the implicit summation in front of the integrator.

Math block `M1` contains four multipliers (each with two implicit summing inputs, one output). The four remaining outputs of this block are used as identity elements[1]. They can be used for more flexibility in the connection topology.

[1] More precise: The first 4 inputs, i.e., the inputs of the first two multipliers in the M1 block are routed to the last 4 outputs of the M1 block - not interferring with the multiplication function.

All in all, one LUCIDAC contains eight integrators and four multipliers.

2.4 Circuit Configuration

Each LUCIDAC is identified by its *MAC address*, which serves as the unique device identifier. The hardware is organized hierarchically as *entities*: a **Carrier** (identified by its MAC address) contains one or more **Clusters**, each of which contains the familiar blocks (M0, M1, U, C, I, etc.) and their individual computing elements.

In the LUCIDAC terminology, there is an important distinction between *specification* and *configuration*:

Specification describes the static structure of the hardware — which entities exist, their types, and their arrangement. A specification is intrinsic to the device and does not change during operation.

Configuration describes the operational settings applied to program a circuit — routing matrices, coefficient values, initial conditions, and ADC assignments. The configuration is volatile and is reset at power-off.

Both are serialized using Google's *Protocol Buffers* (protobuf, see `https://protobuf.dev`), a compact binary serialization format. The resulting files use the `.apb` extension and can contain specification, configuration, or both.

Users typically do not need to work with the protobuf format directly. Instead, circuits are defined programmatically using the lucipy interface (see Chapter 3) and can be serialized for inspection or archival. For offline serialization (without a connected device), the `Circuit` constructor accepts a MAC address directly. When working with a live LUCIDAC, the MAC address is detected automatically via `luci.create_circuit()` (see Chapter 3):

```
from pybrid.lucipy import Circuit
from pybrid.base.proto.io import ProtoIO

c = Circuit(mac="AA-BB-CC-DD-EE-FF")  # offline use
N = c.int(ic=-0.5, slow=True)
c.connect(N, N, weight=0.3)
c.probe(N, adc_channel=0)

pb_file = c.to_config()
ProtoIO.store_module(pb_file.module, "config.apb")
```

The serialized output reflects the entity hierarchy. For instance, the integrator configuration for the above circuit is represented as:

```
items {
  entity { path: "AA-BB-CC-DD-EE-FF/0/M0" }
  itor_config {
    elements { ic: -0.5  k: 100 }
    elements { idx: 1  k: 10000 }
    ...
  }
}
```

where the entity path `AA-BB-CC-DD-EE-FF/0/M0` identifies the M0 block within cluster 0 of the carrier with MAC address `AA-BB-CC-DD-EE-FF`. The integrator element at index 0 is configured with an initial condition of -0.5 and a time scaling factor $k = 100$ (slow mode), while all other integrators use the default $k = 10{,}000$.

To extract the current specification or configuration from a connected device, use the `pybrid` CLI (see also Section 3.3):

```
shell@client $> pybrid lucidac -h <IP> extract --specification -e spec.apb
shell@client $> pybrid lucidac -h <IP> extract --configuration -e config.apb
shell@client $> pybrid read-apb spec.apb
```

> The protobuf definitions will be made available as open source. For the full protocol specification, refer to the pybrid source code.

2.5 Coupling multiple LUCIDACs together

Generally, digital computers can solve arbitrarily large problems given there is enough time and memory available. In contrast to this, analog computers have to grow linearly with the problem size. The advantage of this is that an analog computer exhibits constant times to solution (see for instance [KÖPPEL et al, 2021] and references therein). Computational circuits in LUCIDACs can grow beyond a single LUCIDAC by making use of the front panel inputs and outputs. This way, a number of LUCIDACs can be connected together, thus forming a larger machine.

Connecting several LUCIDACs to form a larger system poses a *synchronization* problem since control of the IC/OP modes as well as ADC acquisition must happen synchronously across all devices.

2.5.1 Setting up multi-device coupling

Synchronization between LUCIDACs is achieved by connecting the included *flatband cable* to the digital I/O headers on all participating devices. At runtime, pybrid automatically designates one device as the *leader* and all others as *followers*. This synchronizes the IC/OP mode transitions and coordinates data acquisition across all devices with minimal jitter. The digital ports should be connected when all participating devices are powered off.

For exchanging *analog signals* between devices, the MCX connectors on the front panel are used. The eight analog outputs and inputs allow routing signals between LUCIDACs. These physical connections determine the analog coupling topology and must be established manually with MCX-MCX cables.

2.5.2 Programming a multi-LUCIDAC circuit

The following example demonstrates four coupled harmonic oscillators distributed across two LUCIDACs. Each LUCIDAC computes two oscillators, and the analog I/O ports are used to exchange signals between the devices. This requires connecting analog outputs 0–3 of one LUCIDAC to the corresponding analog inputs 0–3 of the other, and vice versa.

```
from pybrid.lucipy import LUCIDAC, time_series

# Auto-detect all connected LUCIDACs in the network
# Alternatively, pass multiple IP addresses as strings,
```

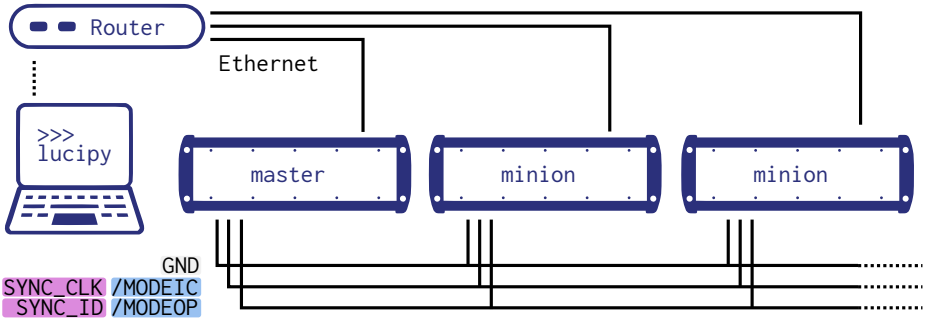


Figure 2.4: Combining multiple LUCIDACs into a larger system requires digital connections (flatband cable to pin headers) as well as ethernet connections. The analog interconnections (MCX-MCX) are not shown.

```
# he first LUCIDAC will be the leader
stack = LUCIDAC()

# Create circuits for each device (index 0 and 1)
l0 = stack.create_circuit(0)
l1 = stack.create_circuit(1)
```

The call to `LUCIDAC()` without arguments auto-discovers all LUCIDACs in the network. Each call to `create_circuit(n)` creates an independent circuit on device n, with its own set of computing elements.

```
# First LUCIDAC: four integrators coupled to analog I/O
l0_int0, l0_int1 = l0.int(ic=-0.105), l0.int(ic=-0.21)
l0_int2, l0_int3 = l0.int(ic=-0.42), l0.int(ic=-0.84)

l0_in0, l0_in1 = l0.input(port=0), l0.input(port=1)
l0_in2, l0_in3 = l0.input(port=2), l0.input(port=3)
l0_out0, l0_out1 = l0.output(port=0), l0.output(port=1)
l0_out2, l0_out3 = l0.output(port=2), l0.output(port=3)

for itor, inp, out in [(l0_int0, l0_in0, l0_out0),
                       (l0_int1, l0_in1, l0_out1),
                       (l0_int2, l0_in2, l0_out2),
                       (l0_int3, l0_in3, l0_out3)]:
    l0.connect(itor, out)     # output integrator to front panel
    l0.connect(inp, itor)     # feed input from other LUCIDAC
```

Each integrator's output is routed to an analog output port, and its input is fed from the corresponding analog input port. On the second LUCIDAC, the same structure is used but with inverted output weights to create the feedback required for oscillation:

```
# Second LUCIDAC: same structure, inverted output weights
l1_int0, l1_int1 = l1.int(ic=-0.105), l1.int(ic=-0.21)
l1_int2, l1_int3 = l1.int(ic=-0.42), l1.int(ic=-0.84)

l1_in0, l1_in1 = l1.input(port=0), l1.input(port=1)
l1_in2, l1_in3 = l1.input(port=2), l1.input(port=3)
l1_out0, l1_out1 = l1.output(port=0), l1.output(port=1)
l1_out2, l1_out3 = l1.output(port=2), l1.output(port=3)

for itor, inp, out in [(l1_int0, l1_in0, l1_out0),
                       (l1_int1, l1_in1, l1_out1),
                       (l1_int2, l1_in2, l1_out2),
                       (l1_int3, l1_in3, l1_out3)]:
    l1.connect(itor, out, weight=-1)  # inverted
    l1.connect(inp, itor)
```

The `weight=-1` on the second LUCIDAC's outputs creates the sign inversion needed to close the oscillator feedback loops across the two devices. Finally, we configure probes, data acquisition, and execute:

```
# Probe all integrators on both devices
for i, itor in enumerate([l0_int0, l0_int1, l0_int2, l0_int3]):
    l0.probe(itor, adc_channel=i)
for i, itor in enumerate([l1_int0, l1_int1, l1_int2, l1_int3]):
    l1.probe(itor, adc_channel=i)

# Configure and run -- applies to all devices in the stack
stack.set_daq(sample_rate=100_000)
stack.set_run(op_time=10_000_000)
run = stack.run()
```

The calls to `set_daq()`, `set_run()`, and `run()` operate on the entire stack. Pybrid automatically handles leader/follower assignment, synchronized mode transitions, and coordinated data acquisition across all devices.

2.6 Coupling THAT with LUCIDAC

The Analog Thing (THAT) is an entry-level classical analog computer with a patch panel (open source/open hardware by anabrid, see https://the-analog-thing.org for details). Connecting THAT and LUCIDAC can be useful for some applications which profit from the patch panel and the interactivity provided by THAT. However, keep in mind that THAT computing elements have lower bandwidth and resolution, compared to LUCIDAC.

The master/minion mode is similar to coupling two or more *The Analog Thing*s (THATs). Connecting a THAT to a LUCIDAC is simple from a digital point of view since the logic levels are compatible. In order to couple a LUCIDAC (digital port) to a THAT (hybrid port), three connections are required (GND, IC and OP). See section 6.1 for the LUCIDAC digital front pinout and for instance https://anabrid.com/that-hybrid-port-desc.pdf for a description of the THAT hybrid port.

For connecting analog signals, the four THAT RCA jacks can be used. THAT has up to four analog I/O signals, labelled X, Y, Z and U. The physical connection is easily realized, for instance, with an RCA-BNC adapter and the supplementary BNC-MCX cables of LUCIDAC. Note that LUCIDAC front panel analog signals are directional. Therefore, when interconnecting two LUCIDACs it is obvious that two inputs or two outputs should not be short-circuited. In contrast, on THAT, the direction of the RCA jacks is dictated by the connections made on its patch panel.

Note that THAT RCA jack levels are ± 1V while LUCIDAC MCX levels are ± 2V. Thus using analog values outside of the interval $[-0.5, 0.5]$ on the LUCIDAC side feeding the THAT will result in erroneous values on the THAT. However, misuse in terms of out of range values will not damage either LUCIDAC or THAT.

> The connection and operation of The Analog Thing (THAT) and LUCIDAC are performed entirely at the user's own risk. Due to the variation in user-defined patching and signal routing, anabrid disclaims any and all warranties and responsibility for the resulting system's performance, stability, or potential for damage arising from this interconnection, including but not limited to signal mismatch, improper voltage levels, or faulty connections. Users must ensure compliance with all specifications, including voltage level differences .., and accept full responsibility for any outcomes.

Section 3

Tutorial: A first model on the LUCIDAC

This section offers a simple entry into the process of "programming" an analog computer given a physical model. While the examples in chapter 4 demonstrate more complex applications, this tutorial provides a step-by-step guide to creating your first circuit on the LUCIDAC. For a more principled and theoretical approach to analog computing, please refer to [ULMANN, 2023/2].

3.1 Connecting to your LUCIDAC

Before you can interact with a LUCIDAC—either through the Python interface or the CLI—you need to know its IP address. The simplest way to find it is to use the `pybrid detect` command, which discovers LUCIDAC devices on your local network using mDNS:

```
shell@client $> pybrid detect
```

This will list all LUCIDAC devices found, along with their IP addresses. Once you know the IP address, set the `LUCIDAC_ENDPOINT` environment variable so that all pybrid tools and Python scripts can find your device:

```
shell@client $> export LUCIDAC_ENDPOINT="tcp://192.168.1.234:5732"
```

> After setting the environment variable, restart your shell or source your profile for the change to take effect. Alternatively, you can pass the IP address directly in Python code via `LUCIDAC("192.168.1.234")` or use the `-h` flag with CLI commands (e.g., `pybrid lucidac -h 192.168.1.234 run`).

3.2 Using the lucipy-interface from Python

In this tutorial, we will implement a simple model for *radioactive decay*[1]. This example is ideal for a first encounter with analog computing as it involves only one integrator and demonstrates the fundamental concept of feedback.

3.2.1 The mathematical model

The radioactive decay process can be described by a first-order ordinary differential equation (ODE) with parameters $\lambda \in (0, 1)$ representing the decay rate and $N_0 \in [0, 1)$ as the initial quantity:

$$\dot{N} = -\lambda N, \quad N(0) = N_0$$

where t is the implicit time variable and $N(t)$ represents the quantity of radioactive material at time t. The dot notation $\dot{N}$ denotes the time derivative $\frac{dN}{dt}$.

We will now show how to transform this computational model into a circuit running on the LUCIDAC system.

3.2.2 Setting up the circuit

First, import the relevant classes from pybrid, connect to the LUCIDAC, and create a new circuit. We also define the parameters for our model:

```
from pybrid.lucipy import Circuit, LUCIDAC, time_series

# Model parameters
_lambda = 0.3       # decay rate
_n0 = 0.8           # initial quantity

# Connect to LUCIDAC and create a circuit
luci = LUCIDAC()
```

[1] See also https://the-analog-thing.org/THAT_First_Steps.pdf, page 15 for a similar example on a different analog computing platform.

```
c = luci.create_circuit()
```

3.2.3 Identifying required computing elements

As we analyze the ODE, we observe that the highest-order derivative present is $\dot{N}$, a first-order derivative. This means we require exactly one integrator to solve this equation. The integrator will compute $N(t)$ by integrating its input signal over time.

We create an integrator whose output represents N:

```
N = c.int(ic = -_n0, slow=True)
```

Note two important aspects of this line of code:

1. The integrator's output is N, which means its input must be $\dot{N}$.
2. Due to the inversion property of LUCIDAC integrators (see section 2.3), the initial condition needs to be inverted. Hence we pass `-_n0` rather than `_n0`.

The parameter `slow=True` deserves special attention: By default, LUCIDAC uses a time scaling factor of $k_0 = 10{,}000$, meaning that circuits are executed 10,000 times faster than in real time. For this introductory example, we use `slow=True` to reduce this factor to $k_0 = 100$, slowing down the computation. This allows us to observe a smoother curve during data acquisition and makes the system more suitable for visualization on a DSO.

3.2.4 Constructing the feedback loop

Now we need to configure the integrator's input signal. According to our ODE, $\dot{N}$ is defined by the expression $-\lambda N$. This right-hand side (RHS) of the equation must be fed to the integrator as its input signal.

We construct this input signal by taking the output of the integrator (N), multiplying it by λ using a coefficient, and feeding it back to the integrator's input. Note that we multiply by λ (not $-\lambda$) because the integrator's inversion property automatically accounts for the negative sign:

```
c.connect(N, N, weight = _lambda)
```

This practice of connecting the output of an integrator to its own input is generally called *feedback* and is a fundamental technique in analog computing. Under the hood, this single line of code automatically configures the M0, U, C, and I blocks of the LUCIDAC such that the signal is routed as required through the UCI interconnection matrix.

3.2.5 Configuring data acquisition

To observe the results of our computation, we need to specify which signals should be sampled through the internal ADCs. Since we are interested in the evolution of $N(t)$, we connect N to the first ADC channel:

```
c.probe(N, adc_channel=0)
```

3.2.6 Running the computation

The circuit definition is now complete. We now configure the execution parameters. We will integrate for 0.5 seconds of wall-clock time, which with our time scaling factor $k_0 = 100$ corresponds to 50 seconds of "equation time" (i.e., the variable t in the ODE ranges from 0 to 50):

```
op_secs     = 0.5
sample_rate = 100_000

luci.set_daq(num_channels=1, sample_rate=sample_rate)
luci.set_run(ic_time=1_000, op_time=int(op_secs * 1_000_000_000))
```

Here, we set the sample rate to 100,000 samples per second, providing high temporal resolution. The `ic_time` parameter specifies how long (in nanoseconds) the integrators should remain in Initial Condition (IC) mode to load their initial values before switching to Operating (OP) mode.

Finally, one line of code is sufficient to trigger the circuit execution:

```
run = luci.run()
```

This method blocks until the computation completes and returns all sampled data points. The returned data structure contains the ADC samples that can be processed and visualized. See

the examples in chapter 4 for demonstrations of how `matplotlib` can be used to create plots from this data.

3.2.7 Expected results

If the run was successful, a plot of the acquired data should show an exponential decay curve as depicted in figure 3.1. The curve should start at approximately $N_0 = 0.8$ and decay toward zero with a characteristic time constant determined by λ.

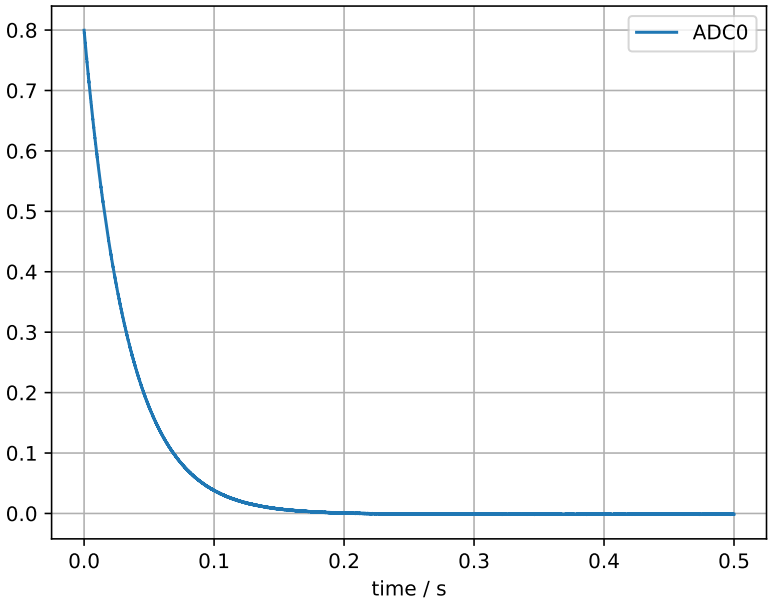


Figure 3.1: Expected output: Exponential decay of radioactive material computed on the LUCIDAC and acquired via internal ADCs

This simple example demonstrates the essential workflow for programming the LUCIDAC: analyzing the mathematical model, identifying required computing elements, setting up connections (including feedback where necessary), configuring data acquisition, and executing the

computation. More complex examples building on these principles can be found in chapter 4.

3.3 Using the pybrid CLI client

While the lucipy Python interface offers maximum flexibility and is ideal for complex hybrid computing applications, pybrid also provides a command-line interface (CLI) for quick interactions with the LUCIDAC system. The CLI is particularly useful for system diagnostics, testing connectivity, and executing pre-defined circuits without writing Python scripts.

3.3.1 CLI commands overview

The pybrid CLI provides several commands for interacting with the LUCIDAC. The following overview covers the most important ones:

`pybrid detect` Discovers LUCIDAC devices on the local network using mDNS and lists their IP addresses and names.

`pybrid lucidac extract` Downloads the device's hardware specification and/or current circuit configuration, storing it as an APB (protobuf binary) file. This is useful for archiving configurations or replicating a circuit setup on another device. Available flags:

```
shell@client $> pybrid lucidac -h <IP> extract --specification -e spec.apb
shell@client $> pybrid lucidac -h <IP> extract --configuration -e config.apb
shell@client $> pybrid lucidac -h <IP> extract --calibration -e calib.apb
```

`pybrid read-apb` Reads an APB file and displays its contents as human-readable text. This is the primary way to inspect protobuf configuration files:

```
shell@client $> pybrid read-apb config.apb
```

`pybrid lucidac run` Executes a computation on the LUCIDAC. Optionally loads a circuit configuration from an APB file:

```
shell@client $>
     pybrid lucidac -h <IP> run -c config.apb --op-time 0.5 --sample-rate 100000
```

Common options include `--op-time` (operating time in seconds), `--sample-rate` (ADC sample rate in Hz), `--ic-time` (initial condition time in nanoseconds), `--output` (export data to file), and `--plot` (display results graphically).

`pybrid proxy` Starts a proxy server that sits between the client and one or more LUCIDACs. The proxy uses a native C++ implementation for fast protocol handling. This is particularly important when the network connection between the client and the LUCIDAC is slow or has high latency: the LUCIDAC streams measurement samples at a constant rate, and if the client cannot receive them fast enough, samples are lost due to buffer overflow.

The solution is to run the proxy on a machine close to (or on the same network as) the LUCIDAC, and then connect pybrid to the proxy:

```
shell@client $> # On the machine close to the LUCIDAC:
shell@client $> pybrid proxy -b 192.168.1.234
shell@client $>
shell@client $> # On the client machine, connect to the proxy:
shell@client $> export LUCIDAC_ENDPOINT="tcp://<proxy-ip>:5732"
```

`pybrid report` Generates a hardware calibration and test report as a PDF file. The report includes oscillator tests, lane accuracy tests, multiplier validation, and integrator checks. This is useful for diagnostics and when working with anabrid support:

```
shell@client $> pybrid report report.pdf
```

3.3.2 Extracting specifications and configurations

A powerful workflow for working with LUCIDAC circuits involves extracting and replaying configurations. After setting up a circuit programmatically (via lucipy) and executing it, you can extract the resulting configuration from the device:

```
shell@client $>
     pybrid lucidac -h <IP> extract --specification --configuration -e full_state.apb
```

This APB file captures the complete device state and can be inspected with:

```
shell@client $> pybrid read-apb full_state.apb
```

The extracted configuration can later be loaded back onto the device using the `run` command, effectively replaying the circuit:

```
shell@client $> pybrid lucidac -h <IP> run -c full_state.apb
```

This workflow is useful for reproducing experiments, sharing circuit configurations between users, or archiving setups for later use.

Section 4

Example applications

This section shows example applications for *lucipy* client side programming.

How to run an example

For the sake of brevety, all examples are written without explicit LUCIDAC endpoints. That is, we write `LUCIDAC()` instead of, e.g. `LUCIDAC("tcp://192.168.1.234:5732")`. Conventionally, without a given endpoint lucipy will test for the `LUCIDAC_ENDPOINT` environment variable. If not given, it will carry out an auto-detection in order to find LUCIDAC hardware to run the commands on. If there are multiple LUCIDAC systems in a subnet, explicit endpoints will be required to address individual systems. Consult section 7.1 in case of questions how to build up a connection to the system. The simplest way to reproduce the examples on your system is a setup as proposed in figure 1.1 on page 8. Put the oscilloscope in automatic or trigger mode and just execute an example file.

The shortest way to do so from scratch, using not more than an installed Python ≥ 3.11[1]:

[1] Note that we assume a Linux setup here with a usable shell (e.g., bash) and the uv package manager (https://docs.astral.sh/uv/), but this setup usually applies 1:1 to different operating systems as well

```
shell@client $> curl -LsSf https://astral.sh/uv/install.sh | sh  # install uv
shell@client $> uv venv --python 3.13
shell@client $> uv pip install pybrid-computing
shell@client $> git clone https://github.com/anabrid/lucidac-resources.git
shell@client $> cd lucidac-resources/examples/
shell@client $> export LUCIDAC_ENDPOINT="tcp://192.168.123.123"  # if applicable
shell@client $> python lorenz.py
```

In case you do not know the LUCIDAC's IP, you may use autodiscover by *neither* specifying an `LUCIDAC_ENDPOINT` nor passing an address to the `LUCIDAC()` constructor.

4.1 Lorenz attractor

This example shows the *Lorenz system*. This is a set of three coupled first order differential equations for the variables $x(t), y(t), z(t)$. The code presented implements a rescaled version as in https://analogparadigm.com/downloads/alpaca_2.pdf. A circuit is shown in figure 4.1. The system has chaotic behaviour and the (x, y) evolution and phase space computed by lucidac is shown in figure 4.2.

Example listing: **lorenz.py**

```python
# Copyright (c) 2022-2025 anabrid GmbH
# Contact: https://www.anabrid.com/licensing/
# SPDX-License-Identifier: MIT OR GPL-2.0-or-later
"""
Lorenz Attractor

This example implements the Lorenz attractor on the LUCIDAC.

Reference: Analog Paradigm Application Note 2
https://analogparadigm.com/downloads/alpaca_2.pdf
"""

from pybrid.lucipy import Circuit, LUCIDAC
import matplotlib.pyplot as plt
import numpy as np


###
# Create a simple circuit in lucipy-syntax
###

###
# Auto-detect LUCIDAC-device (empty constructor) or:
# - set environment variable LUCIDAC_ENDPOINT to a connection string
# - pass the connection string directly
#
```

```
# where the connection string is 'tcp://<LUCIDAC IP or hostname>:5732'.
###
luci    = LUCIDAC()

l   = luci.create_circuit()             # Create a circuit

a   = 1.0
b   = 2.8
c   = 2.666 / 10

mx  = l.int()                           # Integrators with initial condition
my  = l.int()
mz  = l.int(ic = .3)
xz  = l.mul()
xy  = l.mul()

l.connect(mx, xz.a)                     # Product -x * -z = xz
l.connect(mz, xz.b, weight = 2)

l.connect(mx, xy.a)                     # Product -x * -y = xy
l.connect(my, xy.b)

l.connect(my, mx, weight = -a)
l.connect(mx, mx, weight = +a)

l.connect(mx, my, weight = -b)
l.connect(xz, my, weight = -5)
l.connect(my, my, weight = .1)

l.connect(xy, mz, weight = 2.5)
l.connect(mz, mz, weight = c)

l.probe(mx, adc_channel = 0)            # Connect multiplier/integrator to ADC
                                        # to sample data
l.probe(my, adc_channel = 1)
l.probe(mz, adc_channel = 2)

# Analog output: uncomment to output the x, y, z signals on Analog Outputs
# 0, 1, 2
# l.probe(mx, front_port=0)
# l.probe(my, front_port=1)
# l.probe(mz, front_port=2)

###
# Settings for smapling and cirucit execution
###
op_secs     = .1                        # duration of OP cycle in seconds
sample_rate = 100_000                   # samples per second (max: 150_000 for each
     channel)

luci.set_daq(num_channels=3, sample_rate=sample_rate)
luci.set_run(ic_time = 1_000, op_time=int(op_secs * 1_000_000_000))

###
# Run circuit and start sampling
###
```

```
82  run = luci.run()
83
84  ###
85  # Receive sample data and plot
86  ###
87  ax = plt.figure().add_subplot(projection='3d')
88  ax.plot(*np.array(run.data), ls="-", marker="+", markersize=1.5)
89  ax.set_xlabel("X")
90  ax.set_ylabel("Y")
91  ax.set_zlabel("Z")
92  plt.show()
```

Executing this example as shown above opens a 3D plot showing the Lorenz attractor as sampled from the LUCIDACs execution. Note that what is displayed is the result of the *DAQ* process. The output signals may be output via the analog outputs as well by uncommenting lines 66-68 and connecting a DSO to the three analog outputs from the left. By conneting one input of the DSO to the LUCIDAC's OP output, it is possible to *trigger* the DSO data acquisition process on the start of the LUCIDAC's computation (represented as a *falling* flank of the OP signal).

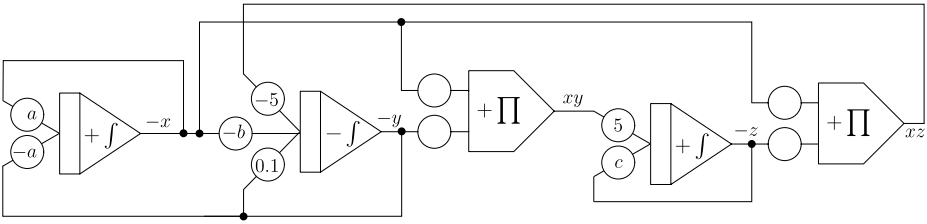


Figure 4.1: Lorenz circuit, empty potentiometers are weight$= 1$

Figure 4.2: Example display on an attached oscilloscope of the Lorenz attractor

4.2 Rössler attractor

The Rössler attractor is another nonlinear and chaotic ODE example. The code presented here is equivalent to the scaled version presented in https://analogparadigm.com/downloads/alpaca_1.pdf. See figure 4.3 for the output of this program on a digital oscilloscope.

Example listing: **roessler.py**

```
# Copyright (c) 2022-2025 anabrid GmbH
# Contact: https://www.anabrid.com/licensing/
# SPDX-License-Identifier: MIT OR GPL-2.0-or-later

from pybrid.lucipy import Circuit, LUCIDAC
import matplotlib.pyplot as plt
import numpy as np

###
# Create a Roessler attractor circuit in lucipy-syntax
###

###
# Auto-detect LUCIDAC-device (empty constructor) or:
# - set environment variable LUCIDAC_ENDPOINT to a connection string
# - pass the connection string directly
#
# where the connection string is ‘tcp://<LUCIDAC IP or hostname>:5732‘.
###
luci    = LUCIDAC()

r   = luci.create_circuit()             # Create a circuit

x   = r.int(ic = .0066)                 # Integrators with initial conditions
y   = r.int()
z   = r.int()
m   = r.mul()                           # Multiplier for nonlinear term
c   = r.const()                         # Constant source

r.connect(y, x, weight = -0.8)
r.connect(z, x, weight = -2.3)

r.connect(x, y, weight = 1.25)
r.connect(y, y, weight = -0.2)

r.connect(c, z, weight = +0.005)
r.connect(m, z, weight = +5.)           # Multiple connections to amplify
r.connect(m, z, weight = +5.)
r.connect(m, z, weight = +5.)

r.connect(z, m.a, weight = -1)          # Compute nonlinear term
r.connect(x, m.b)
r.connect(c, m.b, weight = -0.3796)

r.probe(x, adc_channel=0)               # Connect integrators to ADC
```

```
r.probe(y, adc_channel=1)               # to sample data
r.probe(z, adc_channel=2)

###
# Settings for sampling and circuit execution
###
op_secs     = .1                        # Duration of OP cycle in seconds
sample_rate = 100_000                   # Samples per second (max: 150_000 for each
     channel)

luci.set_daq(num_channels=3, sample_rate=sample_rate)
luci.set_run(ic_time = 1_000, op_time=int(op_secs * 1_000_000_000))

###
# Run circuit and start sampling
###
run = luci.run()

###
# Receive sample data and plot
###
ax = plt.figure().add_subplot(projection='3d')
ax.plot(*np.array(run.data), ls="-", marker="+", markersize=1.5)
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
plt.show()
```

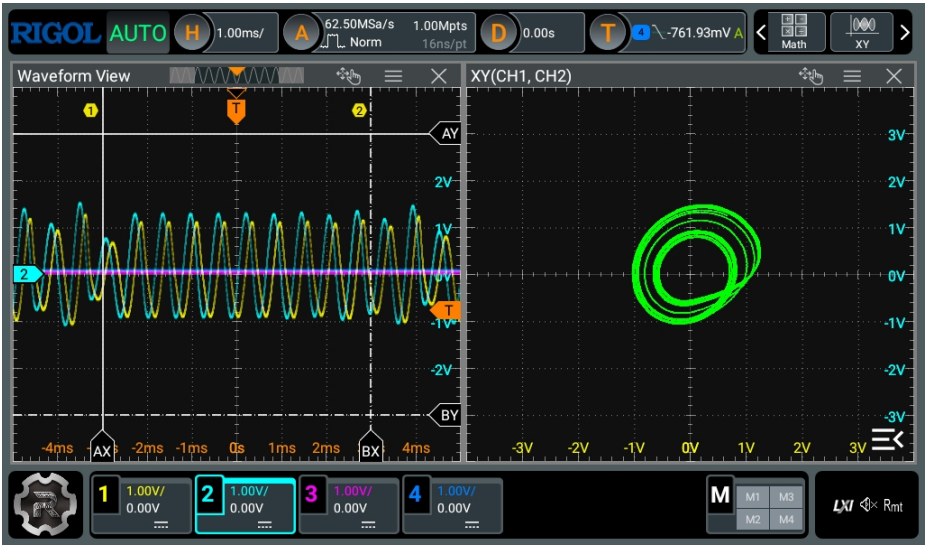


Figure 4.3: Example display of the Roessler attractor (output of previous listing)

4.3 Hindmarsh Rose Neuronal Bursting

This example implements a rather intricate mathematical model of neural bursting and spiking due to Hindmarsh and Rose, see https://analogparadigm.com/downloads/alpaca_28.pdf. The output is shown in figure 4.4.

Example listing: **hindmarsh_rose.py**

```
# Copyright (c) 2022-2025 anabrid GmbH
# Contact: https://www.anabrid.com/licensing/
# SPDX-License-Identifier: MIT OR GPL-2.0-or-later
"""
Hindmarsh-Rose Neuron Model

This example implements the Hindmarsh-Rose neuron model on the LUCIDAC.

Reference: Analog Paradigm Application Note 28
https://analogparadigm.com/downloads/alpaca_28.pdf
"""

from pybrid.lucipy import Circuit, LUCIDAC, time_series
import matplotlib.pyplot as plt
import numpy as np


###
# Create a Hindmarsh-Rose neuron model circuit in lucipy-syntax
###

###
# Auto-detect LUCIDAC-device (empty constructor) or:
# - set environment variable LUCIDAC_ENDPOINT to a connection string
# - pass the connection string directly
#
# where the connection string is 'tcp://<LUCIDAC IP or hostname>:5732'.
###
luci    = LUCIDAC()

hr      = luci.create_circuit()         # Create a circuit

x   = hr.int(ic = +1.)                  # Integrators with initial conditions
y   = hr.int(ic = -1.)
z   = hr.int(ic = +1, slow = True)      # Slow dynamics for adaptation
x2  = hr.mul()                         # Multipliers for nonlinear terms
x3  = hr.mul()
c   = hr.const()                        # Constant source

hr.connect( x, x2.a)                    # Compute x^2
hr.connect( x, x2.b)

hr.connect( x, x3.a)                    # Compute x^3
hr.connect(x2, x3.b)

```

```
hr.connect(x3, x, weight = +4.)         # Fast subsystem (membrane potential)
hr.connect(x2, x, weight = +6.)
hr.connect( y, x, weight = +7.5)
hr.connect( z, x)
hr.connect( c, x)

hr.connect(x2, y, weight = 1.333)       # Recovery variable
hr.connect( y, y)
hr.connect( c, y, weight = -0.066)

hr.connect( x, z, weight = -0.4)        # Slow adaptation current
hr.connect( c, z, weight = +0.32)
hr.connect( z, z, weight = +0.1)

hr.probe(x, adc_channel=0)              # Connect integrators to ADC
hr.probe(y, adc_channel=1)              # to sample data
hr.probe(z, adc_channel=2)

###
# Settings for sampling and circuit execution
###
op_secs     = .2                        # Duration of OP cycle in seconds
sample_rate = 100_000                   # Samples per second (max: 150_000 for each
     channel)

luci.set_daq(num_channels=3, sample_rate=sample_rate)
luci.set_run(ic_time = 1_000, op_time=int(op_secs * 1_000_000_000))

###
# Run circuit and start sampling
###
run = luci.run()

###
# Receive sample data and plot
###
for ix, values in enumerate(run.data):
    x = time_series(sample_rate, len(values))
    plt.plot(x, [-t for t in values], label=f"Probe␣{ix}")
plt.xlabel("time␣/␣s")
plt.legend()
plt.grid()
plt.show()
```

Figure 4.4: Example display of the Hindmarsh Rose model (output of previous listing)

4.4 Van der Pol oscillator

This example implements a classic amplitude stabilized oscillator first described by VAN DER POL. See figure 4.5 for the qualitative result.

Example listing: **vdp.py**

```
# Copyright (c) 2022-2025 anabrid GmbH
# Contact: https://www.anabrid.com/licensing/
# SPDX-License-Identifier: MIT OR GPL-2.0-or-later
from pybrid.lucipy import Circuit, LUCIDAC, time_series
import matplotlib.pyplot as plt
import numpy as np

###
# Create a Van der Pol oscillator circuit in lucipy-syntax
###

###
# Auto-detect LUCIDAC-device (empty constructor) or:
# - set environment variable LUCIDAC_ENDPOINT to a connection string
# - pass the connection string directly
#
# where the connection string is ‘tcp://<LUCIDAC IP or hostname>:5732‘.
###
luci    = LUCIDAC()

vdp = luci.create_circuit()             # Create a circuit

eta = 4                                 # Nonlinearity parameter

mdy = vdp.int()                         # Integrators
y   = vdp.int(ic = 0.1)
y2  = vdp.mul()                         # Multipliers for nonlinear terms
fb  = vdp.mul()
c   = vdp.const()                       # Constant source

vdp.connect(fb, mdy, weight = -eta)
vdp.connect(y,  mdy, weight = -0.5)

vdp.connect(mdy, y, weight = 2)

vdp.connect(y, y2.a)                    # Compute y^2
vdp.connect(y, y2.b)

vdp.connect(y2,  fb.a, weight = -1)     # Build nonlinear feedback term
vdp.connect(c,   fb.a, weight = 0.25)
vdp.connect(mdy, fb.b)

out0 = vdp.output(0)                    # Allocate output MCX plugs
out1 = vdp.output(1)

vdp.connect(mdy, out0)                  # Connect signals to front panel outputs
```

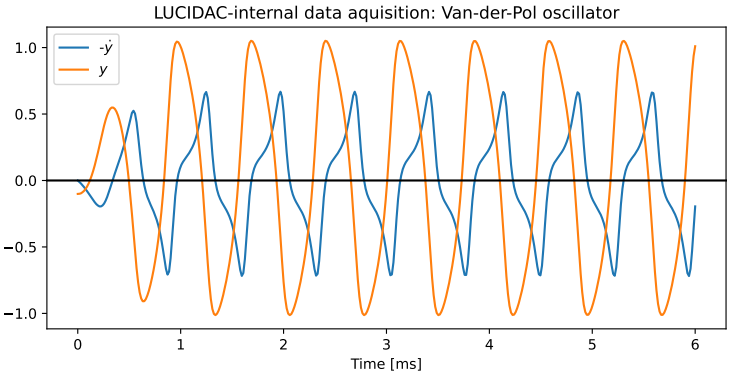


Figure 4.5: Van der Pol oscillator evolution acquired with ADCs, plotted with pyplot

```
47  vdp.connect(y,   out1)
48
49  vdp.probe(mdy)                          # Connect to ADC to sample data
50  vdp.probe(y)
51
52  ###
53  # Settings for sampling and circuit execution
54  ###
55  op_secs     = 0.01                      # Duration of OP cycle in seconds
56  sample_rate = 100_000                   # Samples per second (max: 150_000 for each
        channel)
57
58  luci.set_daq(num_channels=2, sample_rate=sample_rate)
59  luci.set_run(ic_time = 1_000, op_time=int(op_secs * 1_000_000_000))
60
61  ###
62  # Run circuit and start sampling
63  ###
64  run = luci.run()
65
66  ###
67  # Receive sample data and plot
68  ###
69  ax = plt.figure().add_subplot()
70  ax.plot(*np.array(run.data), ls="-", marker="+", markersize=1.5)
71  ax.set_xlabel("X")
72  ax.set_ylabel("Y")
73  plt.show()
```

Section 5

Device administration and maintenance

LUCIDAC is primarily meant to be used in a IPv4 network. From an administration point of view, this raises a number of issues, in particular about usage, monitoring, maintenance and access control. These topics are covered in this section.

5.1 Detection and monitoring utilities in `pybrid`

The `pybrid` CLI client used in the Tutorial in Section 3.3 supports maintenance functions besides running circuits:

- `$> pybrid detect` : Detect LUCIDACs within the same network by broadcast and list their names and IP addresses
- `$> pybrid lucidac -h <IP> display` : Display the hardware structure of the computer, including information on the MAC addresses
- `$> pybrid lucidac -h <IP> get-system-temperatures` : Displays temperatures for individual elements within the block. Even though the LUCIDAC is only passively cooled, it generally manages to keep its temperature even after long-running computations.

When operating multiple LUCIDACs within the same network, the parameter `-h <IP>` lets you select LUCIDACs by their IP address. For more information, please use `pybrid lucidac --help`.

5.2 Device identification

From the vendor point of view, the following information is imprinted onto the microcontroller flash memory: Serial number (a short integer), Serial UUID, Manufacturer name, default user and admin passwords. This information is assigned by anabrid at fabrication time. Furthermore, the microcontroller holds its one-time-programmed MAC address and USB Serial ID. This information is assigned by the MCU fabrication. All this information is also printed onto the serial plate stickers on the back of the device (cf. figure 1.3 on page 10 as well as the front matter of this booklet).

5.3 Firmware Update

We believe that for the sake of reproducibility of results, predictivity of the device, and availability in critical situations, devices should never autonomously and unannounced run a software update on their own. We also believe devices should never "call home" on their own. The ways of searching for a new firmware image and its instsallation are presented below.

The embedded microcontroller in the LUCIDAC system runs either the open sourced firmware (initial stack) available at `http://github.com/anabrid` or the most recent version of the mainline stack at `https://github.com/anabrid/lucidac-resources`. Releases with binary builds ready to be flashed on the LUCIDAC MCU are provided there. For an update, first connect your LUCIDAC via USB and build `tycmd` [1] according to the instructions or install it directly from your distribution (e.g. `sudo apt-get install tycmd` for Ubuntu). In order to flash the `firmware.hex` file, enter

```
$> tycmd upload firmware.hex
```

In order to check what devices are connected, use `tycmd list`.

In general, it is recommended to update LUCIDAC regularly to profit from bug fixes, stability improvements and new features.

The Teensy MCU cannot be *bricked* since it has an external bootloader. It is also impossible to lock yourself out of the device since the serial connection using the USB port always allows administrator access to the device. For further details and update instructions on the MCU please visit `https://www.pjrc.com/teensy/` or see Section 6 on embedded programming.

[1] https://github.com/Koromix/tytools

5.4 Physical maintenance

Technically, the LUCIDAC device needs *no regular maintenance* (see page 67). The device itself is fully enclosed (it does not need external airflow). Opening the device is *not* part of regular usage and must not be done by untrained personnel. Opening the device without proper ESD protection and gloves can and will damage the hardware!

Although it is easy to open the device by removing the screws holding the front panel in place (the internals can be pulled out on a rail), opening the device will invalidate any warranty claims.

5.5 Testing and calibration

The LUCIDAC device performs self-tests and calibration routines autonomously. This is part of the firmware and happens internally, without user interaction at various times, for instance during startup and before running a computation. Note that it is *not* necessary for the user to calibrate LUCIDAC computing elements. The device comes fully calibrated from the manufacturer. In contrast, certain firmware versions might offer ways to invoke the auto-calibration in order to achieve accuracy goals. In case of self-test or calibration failures, the device will report these with signalling LEDs and entries in the system log.

For advanced users who require fine-grained control over when calibration routines are executed, the pybrid Session API provides programmatic access to calibration scheduling. Please refer to the pybrid documentation at `https://anabrid.github.io/pybrid-computing/` for details on session-based calibration control.

Section 6

Embedded programming

> **Note:** This entire chapter describes the software architecture and embedded programming capabilities of the *initial* open-source stack only. The mainline stack's firmware is not open source and uses a different, protobuf-based architecture. The information below does not apply to the mainline firmware. For more information on the initial stack, see Section 1.8.

Embedded programming means writing code which runs directly on the MCU of the LUCI-DAC. The main reason to do this instead of host based programming is to make use of the real-time capabilities of the MCU, thus allowing the implementation of low-latency analog-digital hybrid algorithms with the following advantages:

1. Saving ethernet or USB latency, which is typically of the order of a few Milliseconds per roundtrip time. This clearly dominates short analog computations which only take a few microseconds to complete.

2. Saving protocol overhead (de-/serialization resources on the MCU). This dominates the digital compute time required for applying circuit configurations.

3. Fine-grained access to the underlying hardware allows advanced techniques inaccessible to the API exposed over the protocol. For instance, custom data acquisition and

online postprocessing of data can be done right on the MCU while results can be fed back into the analog circuitry directly.

Additions or changes in the digital communication with respect to the IP networking stack (such as new TCP servers speaking for instance MQTT), protocol extensions to the existing protocol, as well as low-level digital general purpose I/O on the front panel (speaking for instance SPI to another external device) always require modifications in the firmware.

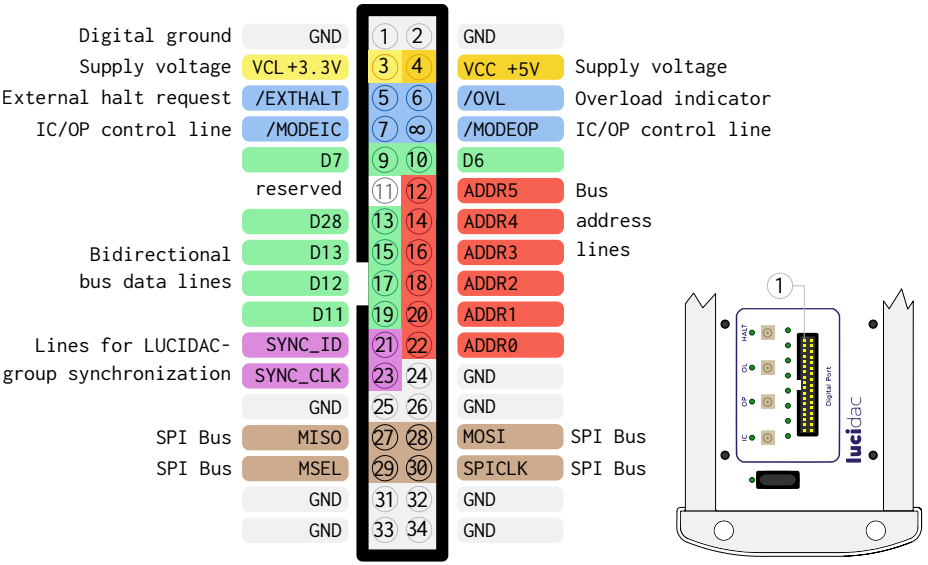


Figure 6.1: Front panel pinout (rotated view from front)

6.1 Front panel digital connector

There is a two pin header on the lower left of the front panel (figure 6.1). In regular use, this port is currently used for inter-LUCIDAC coupling (master/minion mode, cf section 2.5). The front panel digital connector can only controlled by means of custom firmware code, for instance with a compile time plugin.

Several pins are reserved for future applications. The bidirectional data lines can be used to control external equipment while the six address lines are generated by the built-in control module. They also control the SPI bus. The four IC/OP/OVL/HALT lines are also available on MCX ports. These are primarily used as oscilloscope trigger signals but can in principle also be used for controlling other devices (including LUCIDACs).

Care should be taken when connecting external hardware so as not to damage the system. Digital logic levels are 0 V and 3.3 V.

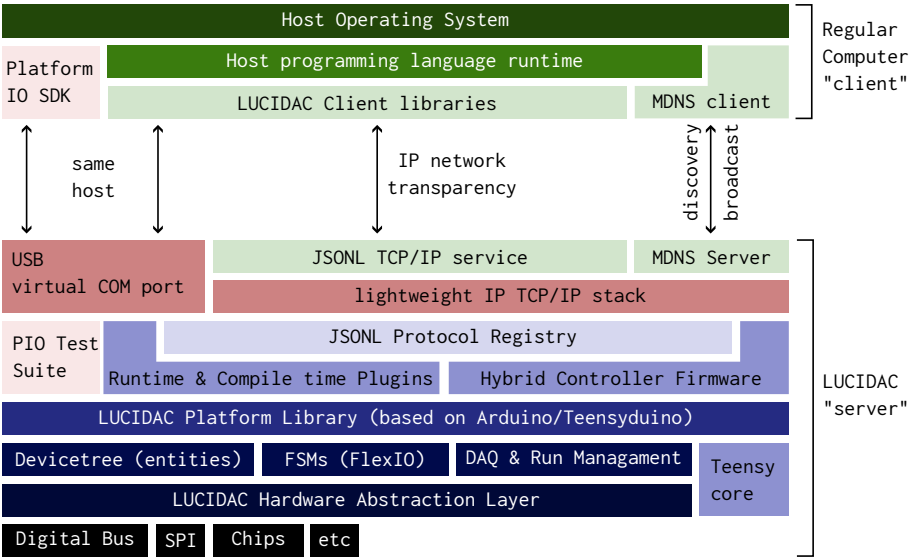


Figure 6.2: LUCIDAC tech stack diagram with focus on communication

6.2 Software architecture and communication interface

The LUCIDAC initial stack's firmware is open-sourced at https://github.com/anabrid/lucidac-firmware. The code is modular/extensible and documented. The documentation is linked in the Github repository. The firmware code is based on Arduino and on the PlatformIO build system (in short *PIO*, see https://platformio.org). Please refer to the firmware documentation for getting started with PlatformIO.

Figure 6.2 shows the full digital interface to the LUCIDAC in a layer/tech stack diagram. The individual layers are explained from bottom to top:

1. On the lowest level, the firmware libraries provide a *hardware abstraction layer* to communicate with the various integrated circuits, their data models, and programming functionality.

2. On the next layer, the firmware exposes the *platform library* which collects business logic that implements the blocks as well as administrative infrastructure for hierarchical hardware management, various finite state machines (FSMs), data acquisition, run management, etc.

3. The low level firmware functionality can be accessed as a library “toolbox” where various functions can be used. This allows writing both the main firmware, custom plugins by the user as well as isolated stand-alone tests of particular parts of the software and hardware using the PlatformIO test system.

4. The Remote Procedure Call (RPC) system is based on a JSON protocol registry, which is accessible independently from the transport layer above.

5. Communication takes place either by means of the Arduino USB Serial terminal or the lightweight IP stack (QNEthernet lwIP).

6. The firmware exposes a number of TCP/IP services, such as the JSONL “native” main server or the QNEthernet-integrated MDNS service announcement.

7. At the ethernet client (or USB host) side, various client codes decode the communication, eventually communicating with the host side and application codes.

> Note that the communication protocol has evolved from JSON to Google’s Protobuf from the initial to the mainline stack. Thus, the intial stack’s versions of the firmware and lucipy are **NOT** compatible with the recent mainline versions.
> For more documentation on the initial stack, refer to the initial version of the handbook.

6.3 System states

The following list provides an overview information about the different states hold by the digital part within the LUCIDAC system. First of all, we differentiate between volatile state (reset at power cycle) and persistent, non-volatile state (stored in EEPROMs/Flash memories and re-read at startup). The following data are non-volatile: Firmware image, networking configuration/user settings, vendor identification information and finally entity data which can be calibration information. Even a reboot of the machine won’t reset the non-volatile information.

The following data are volatile: In general, all analog parts (in particular the reconfigurable interconnection matrix, potentiometer values, initial conditions, overload states), the operating states (IC/OP/HALT state machines), the dynamical runtime ethernet configuration (i.e. DHCP, locking), various software-only subsystem states (such as plugin loaders or ongoing firmware upgrades).

Typically, each subsystem holding volatile information has a reset function on its own. However, a power cycle of the whole system or reboot of the microcontroller will also reset all volatile system states. For further information, please consult the firmware manual.

6.4 Writing a compile time plugin

The firmware is organized as a library "toolbox" exposing several functions. The classical entry point at `src/hybrid_controller.cpp` boils down to the two classical Arduino functions as shown in the following pseudocode:

```
#include "lucidac-firmware-libraries.h"

void setup() {
        setup_firmware();
        // This is where your startup code can be placed, which is executed once
        // at startup.
}

void loop() {
        loop_firmware_services();

        // This is where any code can be placed which will be executed regularly,
        // for instance event loops, services, etc.
}
```

Typically this or similar patterns are referred to as *compile time plugins*. Rudimentary support for *runtime plugins*, which can load and execute machine code at runtime is provided by the firmware. This is will provide a technological demonstrator basis for tightly-coupled co-processor style hybrid computing. (This is where anabrid is heading towards. For further details, please refer to the firmware documentation.)

6.5 Extending the existing network protocol

The LUCIDAC TCP/IP network model is a JSONL-based command-reply protocol and a simple implementation of a remote procedure call (RPC) scheme. In order to extend the protocol with new calls, some experience with C++ is required. What follows is a demonstration listing which allows to start and stop a front panel LED animation running independently in the main loop:

```
#include "lucidac/lucidac.h"
#include "protocol/handler.h"

bool running = false;
int i = 0;

struct MyCustomHandler : public msg::handlers::MessageHandler {
    int handle(JsonObjectConst msg_in, JsonObject &msg_out) override {
        LOG_ALWAYS("My new request is being called");
```

```
        running = msg_in["running"];
    }
};

void setup() {
    // ...
    LOG_ALWAYS("Registering my_request handler...");
    msg::handlers::Registry::get().set("my_request", new MyCustomHandler());
}

void loop() {
    // ...
    auto& lucidac = platform::LUCIDAC::get();
    lucidac.front_panel->leds.set(i, true);
    i = (i+1)%8; // advance
}
```

From lucipy (initial stack), invoking this request is as simple as a plain query, for instance in this interactive application:

```
from lucipy import LUCIDAC
hc = LUCIDAC()
while True:
        # Wait for user input on command line
        input("Press enter to start animation or CTRL+C to stop program")

        # Turn on the LED animation
        hc.query("my_request", {"running": True  })

        # Wait for user input on command line
        input("Press enter to stop  animation or CTRL+C to stop program")

        # Turn off the LED animation
        hc.query("my_request", {"running": False })
```

Section 7

Troubleshooting

This section shows some typical problems which could appear and approaches to solve them. If you have fundamental problems with the device which you cannot solve by yourself, please do not hesitate to send a mail to support@anabrid.com.

7.1 Basic connectivity and startup

System remains dead at startup Check if the Power LED at the front panel is on. If it is not, check the power supply and the position of the power switch on the front panel. After power on, some of the IC/OP/OL/... LEDs should also light up, at least briefly. If this does not happen, your MCU or firmware may be corrupted. All eight status LEDs should be lit within three seconds of powering on. If this does not happen, the fundamental LUCIDAC self-check hardware detection has failed. If the eight LEDs do not turn off again, the firmware has got stuck in the networking setup (waiting for an address, etc.)

System cannot be found in the network Make sure the LEDs on the RJ45 network connector at the back of the device are lit, indicating a physical connection. Make sure the system is properly turned on and all LEDs are off after final startup (wait at least 10 to 20 seconds). If possible, check the log of your local DHCP server for an entry containing the hostname "lucidac". Otherwise, make use of a IP network scanner. There are various ones available for all kind of operating systems, even mobile phones. On linux and Mac, you can use the classical `https://nmap.org/` and scan for the typical LUCIDAC TCP port 5732:

```
shell@client $> nmap -p 5732 192.168.1.0/24 -oG - | grep open
```

In this snippet, insert your local IP network instead of `192.168.1.0`. If this does not work, connect to the USB terminal and check the output for the allocated IP address, which is shown there. This also works with the client codes itself. For instance `lucigo detect` (see section 5 about lucigo) will list USB devices. Pybrid, too, can do this with `pybrid detect`, please refer to the pybrid documentation at https://anabrid.github.io/pybrid-computing/ for further information.

Cannot connect over USB First of all, make sure the USB connection is working. Use only the enclosed USB-A to C cable, which is certified for USB2. In particular, do not use a USB C-to-C cable, because we experienced issues with such cables. Note that LUCIDAC does not support USB-C Power Delivery. Power to LUCIDAC is provided by the enclosed power supply unit. If you connect LUCIDAC via USB and forget to connect the LUCIDAC to its power supply, the device won't even power up.

Check your USB hub structure. Try connecting the LUCIDAC to other USB hubs or ports on your computer. Make sure the device shows up in your operating system. USB recognizes devices, amongst others, by their vendor ID, product ID and their serial number. These are written on the device back plate. Various operating systems support listing the connected devices ("Device Manager" on Microsoft Windows, "System Information app" in Mac OS X Utilities folder, `lsusb` on Linux). Make sure the device shows up there.

USB Serial Terminal doesn't work On Linux, you need to install the udev rules for making sure the virtual serial terminal is registered by the kernel. You find these at https://www.pjrc.com/teensy/00-teensy.rules including the installation guide. Also make sure the access rights of the corresponding device file allow your standard user to connect to the USB serial terminal.

Flashing the firmware doesn't work Normally you should just use the self-contained update procedure provided by the firmware itself. However, if you want to flash the firmware explicitly, for instance because you are developing a modified firmware, the following tips might come in handy:

It is a well-known phenomenon that, depending on the operating system and cables used, sometimes the firmware flasher has to be invoked several times. If the

`$> tycmd` utility does not work well for you, you might also want to try out https://github.com/PaulStoffregen/teensy_loader_cli. See also https://www.pjrc.com/teensy/check_halfkay.html for the Teensy bootloader.

In rare cases, flashing the firmware only works when pressing the button located on the teensy MCU. Since this button is not accessible from the outside, the system must be opened to get access. Please contact anabrid in such a case after you tried rebooting the system.

7.2 Analog programming problems

I can't find a summing element in LUCIDAC programming: The computer performs an implicit sum in the I-block. Accordingly, there are no summers available as explicit computing elements.

System does not compute what was expected: If using lucipy, you might consider to cross-check your circuit configuration using the simulator included in the initial stack's lucipy. This can be as easy as changing the LUCIDAC endpoint to the integrated Emulator, which has the "virtual" endpoint `emu://`. Please refer to the pybrid documentation at https://anabrid.github.io/pybrid-computing/ for further details.

In general note that analog computing is by definition low precision computing. In particular when dealing with highly unstable systems, there can be differences between an idealized simulation and a real analog computation. If you want to dig deeper into these differences, you might want to consider realistic simulations, taking into account transfer functions which model, for instance, the finite bandwidth of the computing elements.

Furthermore, keep in mind that analog computing requires all values to be within a finite domain (the machine unit domain), typically $[-1, +1]$. This system property is called BIBO (bounded-in, bounded-out) and cannot be violated. Proper scaling of the mathematical equations is required before mapping them onto any analog computer. For more information on how to scale a system of coupled differential equations, see the primers such as [Ulmann, 2023].

Missing signals In order to see what LUCIDAC is computing, you either have to use the internal data acquisition system (DAQ/ADCs) and/or an externally connected oscilloscope. This requires appropriate configuration of the device.

In lucipy, this is possible with the `circuit.probe()` and `circuit.measure()` calls to register an external probe or an internal data conversion. Please refer to the pybrid documentation at https://anabrid.github.io/pybrid-computing/ for further details and examples.

Front panel I/O not working The eight front panel outputs/inputs are mapped to the last eight lanes of the 32 UCI matrix lanes. Counting from 0: Front panel in/out 0 maps to lane 24, front panel in/out 1 maps to lane 25, ...up to front in/out 7 maps to lane 31. Keep in mind that these connections are located between the C and I block. Keep also in mind that for using an input jack, you have to define this as an input, for instance by connecting the corresponding object in lucipy to a computing element, which internally triggers an `acl_select` option:

```
from lucipy import Circuit
circ = Circuit()
fp3 = circ.front_panel(3) # front panel I/O 3
m   = circ.mult()         # multiplier 0
circ.connect(fp3, m)      # this connects front panel input 3 to multiplier 0
```

Caution: The front panel connectors are ESD sensitive. Please make sure no excessive voltages are applied. Touch any grounded object before making or breaking connections. Misuse can permanently damage the computing elements, in particular the corresponding lanes (coefficients) and cross lanes (outging lanes, SH block, computing elements).

My circuit is too big, I run out of computing elements Keep in mind that analog computing means that every mathematical operation in a set of equations has to be mapped to a physical computing element. LUCIDAC provides eight integrators, four multipliers, 32 coefficients, and a variable number of implicit summers. The interconnect circuits also imposes some limitations onto a configuration (see section 2 for details). If your circuit is too big (for instance, it requires too many connections), a single LUCIDAC won't be able to solve it. Maybe the circuit can be simplified.

7.3 Frequently asked questions (FAQ)

How to power off the system? Just turn the system off using the power switch at the front panel. This should not be done while writing permanent settings (see Section 5) to the flash, which could corrupt the system state.

How to use the system with multiple persons the same time? LUCIDAC allows multiple clients to connect over the network at the same time. This can easily result in problems if two clients want to run different circuits at the same time. Therefore, the device provides a simple exclusive locking mechanism. For further details, please refer to the lucipy client or firmware documentation.

How to distinguish multiple devices in the network? LUCIDACs are primarily meant to be distinguished by their Ethernet MAC address. It is written on the back plate of the device. Furthermore, the firmware exposes an identify call which allows the front panel LEDs to blink and thus identify a particular device. For further details refer to the documentation of the relevant codes.

How to use my favourite programming language? Client libraries for LUCIDAC are mostly ordinary TCP/IP network client applications using JSON for encoding data structures. It is straightforward to write libraries in various programming languages. Clients are already available for C/C++, Rust, Python, Julia, Go, TypeScript, JavaScript and some of them are already released on Github. Contact anabrid if you want to make use of these codes or start a client in a new programming language. It also might be an option to use language bindings and foreign function interfaces instead.

Can I connect peripherals to the LUCIDAC USB port? By default the Teensy MCU within the LUCIDAC does not operate as a USB host but as a USB device. You can modify the firmware to have it operate differently and thereby, for instance, connect a USB keyboard or USB mouse to the LUCIDAC. However, this is beyond the scope of our firmware. Another option connecting digital peripherals to LUCIDAC is the front facing port, which will require suitable firmware code to "drive" the custom additions.

Is there some compiler which automates the translation from a differential equation to the LUCIDAC? Anabrid has developed a suite of software aimed at users who are not experts in modelling or analog computing. This includes, but is not limited to, a compiler that turns (systems of) ODEs into circuits. That software and other products are currently not available. If you are interested, please contact anabrid directly.

Section 8

Resources and further reading

This booklet provides only an introduction into the LUCIDAC software and hardware ecosystem. Various resources are available to make your explorations and use of LUCIDAC enjoyable and rewarding. The following web references will help you stay up-to-date with all things LUCIDAC and to get in touch with other members of the analog computing community:

- Anabrid LUCIDAC landing page: https://anabrid.com/lucidac
- Anabrid LUCIDAC webshop: https://shop.anabrid.com
- Anabrid codes at Github: https://github.com/anabrid
- Introductory analog computing circuit examples at https://anabrid.com/application-notes and https://the-analog-thing.org/THAT_First_Steps.pdf
- Resources, including new firmware versions, for the LUCIDAC: https://github.com/anabrid/lucidac-resources

For getting started at analog computing in general, you might want to have a look at the following books and papers:

[Ulmann, 2023] Bernd Ulmann, *Analog and Hybrid Computer Programming*, 2nd edition, DeGruyter, 2023

[Ulmann, 2023/2] Bernd Ulmann, *Analog Computing*, 2nd edition, DeGruyter, 2023

[Köppel et al, 2021] Sven Köppel, Bernd Ulmann, Lars Heimann, Dirk Killat, *Using analog computers in today's largest computational challenges*, doi:10.5194/ars-19-105-2021 [arxiv:2102.07268]

EU-Konformitätserklärung

gemäß der Richtlinie 2011/65/EU (RoHS) vom 8.07.2011

CE

Nr: *AN-4-170000-194873*

Hersteller/Bevollmächtigter 1):

anabrid GmbH
Am Stadtpark 3
D-12167 Berlin
E-Mail.: office@anabrid.com

Die alleinige Verantwortung für die Ausstellung dieser Konformitätserklärung trägt der Hersteller (bzw. Installationsbetrieb): anabrid GmbH

Gegenstand der Erklärung: *LUCIDAC - digital programmierbarer Analogcomputer*

Der oben beschriebene Gegenstand der Erklärung erfüllt die Vorschriften der Richtlinie 2011/65/EU des Europäischen Parlaments und des Rates vom 8. Juni 2011 zur Beschränkung der Verwendung bestimmter gefährlicher Stoffe in Elektro- und Elektronikgeräten.

Angewandte harmonisierte Normen insbesondere:

Angewandte sonstige technische Normen und Spezifikationen:

Unterzeichnet für und im Namen von: anabrid GmbH

Ort/Datum der Ausstellung: Berlin / 27.09.2024

Angabe zur Person des Unterzeichners:

Lars Heimann, Geschäftsführer anabrid GmbH
(Name, Position)

Unterschrift:……

anabrid
anabrid GmbH
Am Stadtpark 3
12167 Berlin
www.anabrid.com

1) „Bevollmächtigter“ ist jede in der Union ansässige natürliche oder juristische Person, die von einem Hersteller schriftlich beauftragt wurde, in seinem Namen bestimmte Aufgaben wahrzunehmen;

LUCIDAC LEGAL DISCLAIMER, SAFETY INSTRUCTIONS, WARRANTY, LIABILITY

1. MANUFACTURER AND CONTACT INFORMATION

Company Name: Anabrid GmbH
Address: Am Stadtpark 3, 12167 Berlin, Germany
Contact Information: Phone: +4930629304720, E-Mail: hello@anabrid.com

1.2. PRODUCT IDENTIFICATION

Model Number: LUCIDAC 1.0
Serial Number: Attached to device
Manufacturing Date: Friday Sept 13, 2024

1.3. INTENDED USE

The LUCIDAC is a fully reconfigurable analog computer co-processor. It is intended for solving complex differential equations, running simulations in physics and engineering, and exploring unconventional computing paradigms. Any use beyond the intended scope (e.g., in non-industrial applications or without appropriate safety precautions) is considered misuse.

2. SAFETY INSTRUCTIONS
2.1 GENERAL SAFETY INFORMATION

This manual complies with ANSI Z535.6 for safety message formatting. Read all instructions thoroughly before installation and operation to ensure compliance with safety regulations.

2.2. SAFETY SIGNAL WORDS:

DANGER: Immediate hazard that will result in serious injury or death.
WARNING: Hazard that could result in serious injury or death.
CAUTION: May result in minor or moderate injury.

2.2 SAFETY SYMBOLS

⚠ Warning: High voltage
🛑 Stop: Emergency power shutoff available

2.3 SAFETY PRECAUTIONS

DANGER: Ensure that all power sources are disconnected before installation.

WARNING: Always use the provided housing to prevent electrical contact.
CAUTION: Only operate the device in an environment with proper ventilation and a stable surface.

2.4 EMERGENCY PROCEDURES

Emergency Stop: Press the power switch located on the front panel. Disconnect all cables.
First Aid: If electrical shock occurs, disconnect power immediately and seek medical assistance.

3. TECHNICAL SPECIFICATIONS
3.1 PERFORMANCE

Power Supply: DC 24V, 1A
Max Operating Pressure: 1030 hPa
Max Operating Temperature: 50° Celsius
Dimensions and Weight: 99 x 357 x 184 mm, weight: 2.3 kg

3.2 MATERIALS AND COATINGS

Body Material Aluminum chassis (IT 103520 housing by BOPLA GmbH, Germany)
Profiles: Al Mg Si 0,5, die-cast corners: Zinc alloy Z410, seal: TPE.

4. INSTALLATION
4.1 UNPACKING INSTRUCTIONS

* Upon receiving the device, inspect packaging for any damage.
* Ensure the package includes the LUCIDAC main unit, power supply, and accompanying accessories (e.g., RJ45 Ethernet cable, USB-C console cable).

4.2 SITE REQUIREMENTS

Space Requirements: Ensure a minimum clearance of 10 cm on all sides for proper ventilation.
Power Requirements: Standard 240/120V AC, converted to 24V DC (adapter included).

4.3 INSTALLATION STEPS

Step 1: Place the LUCIDAC unit on a flat, stable surface.
Step 2: Connect the provided power supply to the barrel jack connector.
Step 3: Use the Ethernet cable to connect the LUCIDAC to your network. The device uses a 100Mbit/sec Ethernet interface .
Step 4: If needed, connect the USB-C cable for console access and firmware flashing.
Step 5: Power on the device by connecting it to the electrical outlet.

5. OPERATION

5.1 Control Panel Overview
Power Button: Located at the front.
LED Indicators: 8 user-configurable LEDs, showing system status.
Trigger outputs for IC/OP/Overload detection.
5.2 OPERATING PROCEDURE
Step 1: Power on the LUCIDAC by connecting the power cable.
Step 2: Access the device through its web-based interface or using the PyAnabrid client on Python.
Step 3: Use the software to configure analog computing elements and run your analog computation.

5.3 SHUTDOWN PROCEDURE

(1) Safely terminate all running computations through the software interface. (2) Disconnect the power supply from the wall socket.

6. MAINTENANCE
6.1 ROUTINE MAINTENANCE

MONTHLY: Check the status LEDs to ensure proper system functionality. Inspect power cables and Ethernet connections for wear.

7. DISPOSAL AND RECYCLING

Dispose of the LUCIDAC in accordance with local electronics recycling regulations. Components such as the aluminum chassis and PCBs should be recycled at appropriate facilities.

APPENDICES

Appendix A: Block Diagram (See main text in document).
Appendix B: Electrical Schematic (Available for licensing or on NDA basis upon request).
Appendix C: Warranty Information (Standard 2-year warranty).

APPENDIX C: WARRANTY INFORMATION (STANDARD 2-YEAR WARRANTY)
WARRANTY PERIOD

Anabrid GmbH provides a standard 2-year warranty for the LUCIDAC co-processor starting from the date of purchase.

COVERAGE

The warranty covers defects in materials and workmanship under normal use and service conditions. If a defect arises during the warranty period, Anabrid will either repair or replace the defective product at no additional charge.

EXCLUSIONS

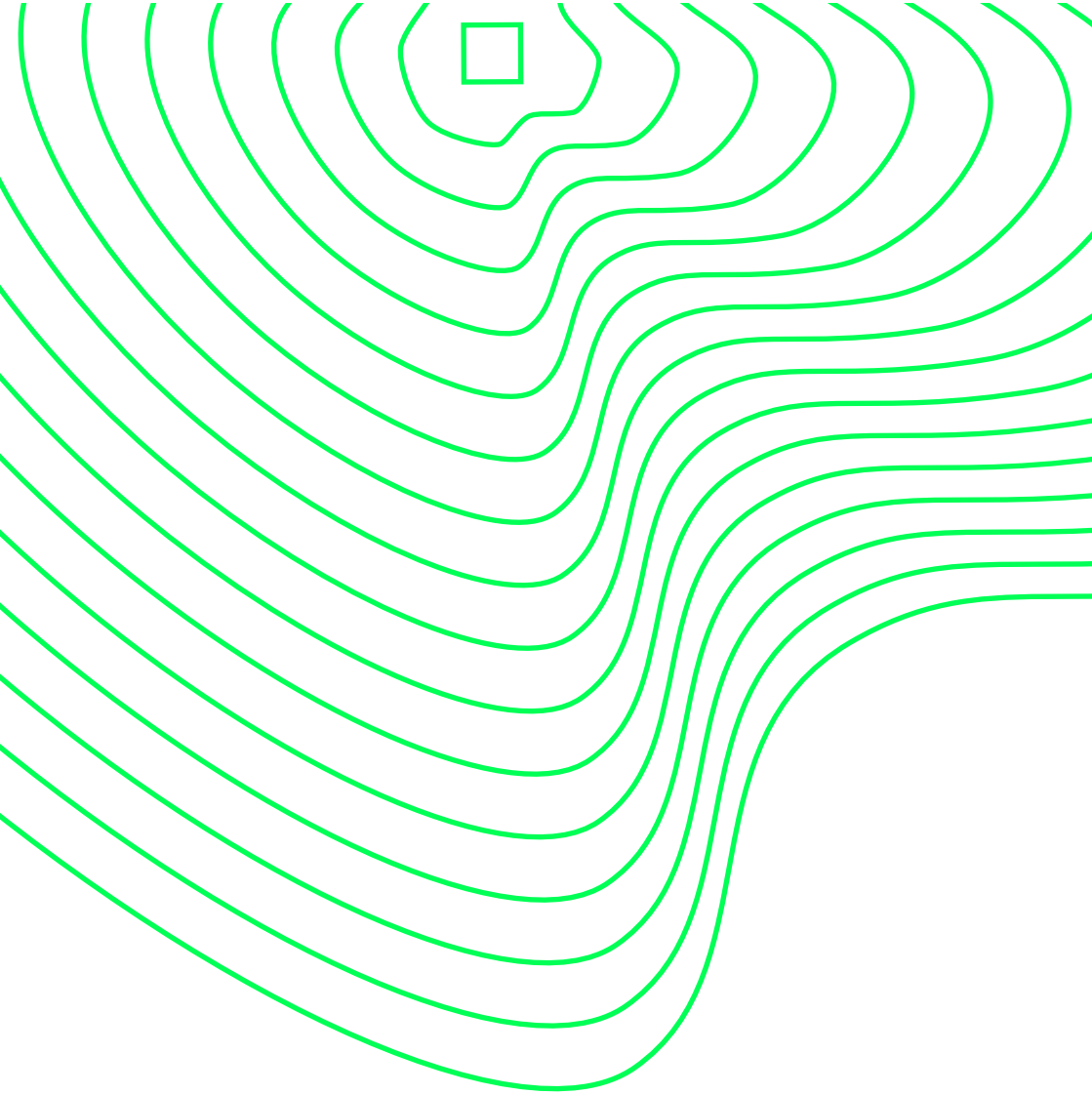
This warranty does not cover:
(1) Damage caused by improper installation, misuse, or unauthorized modifications. (2) Normal wear and tear, cosmetic damage, or accidents. (3) Damage resulting from environmental factors such as excessive heat, humidity, or power surges.

WARRANTY CLAIM PROCESS

0. Contact Anabrid customer support at hello@anabrid.com with proof of purchase and a detailed description of the issue.
1. Upon approval, you will receive instructions for returning the product for inspection.
2. If the defect is confirmed, a repair or replacement will be provided. Anabrid covers shipping costs for repairs or replacements within the warranty period.

LIMITATION OF LIABILITY

Anabrid's liability is limited to the replacement or repair of the defective product. Under no circumstances shall Anabrid be liable for any indirect, incidental, or consequential damages arising from the use or inability to use the product.
For more details, please refer to the full warranty terms on the Anabrid website or contact customer service.



anabrid